



openLOWDIN

Release 0.1

openLOWDIN

May 04, 2026

CONTENTS:

1	About	3
1.1	Capabilities	3
1.2	Documentation	3
1.3	How to cite:	3
1.4	History	4
1.5	Acknowledgements	4
2	Developers and contributors	5
2.1	Active developers (alphabetical order)	5
2.2	Contributors (alphabetic order)	5
3	Installation	7
3.1	Dependencies	7
3.2	Getting the code	7
3.3	Basic installation	7
3.4	Step-by-step installation	8
4	Getting started	11
4.1	Input file	11
4.2	How to run	12
4.3	Scratch	12
5	Publications	13
5.1	Papers	13
5.2	Thesis	14
6	Code	17
7	Input	19
7.1	General structure	19
7.2	GEOMETRY block	19
7.3	TASKS block	20
8	Lib	23
8.1	DataBases	23
8.2	Basis	25
8.3	Potential Basis	27
9	CONTROL	31
9.1	Dummy variables	31
9.2	Geometry optimization	31

9.3	Atomic connectivity	32
9.4	COSMO	32
9.5	Info and units	32
9.6	General	33
9.7	Molecular Mechanics	33
9.8	Miscellaneous options	33
9.9	Integrals transformation	33
9.10	Libraries	34
10	Outputs	35
10.1	Molden and AIM files	35
10.2	Gaussian Cubes	36
10.3	Gnuplot 2D and 3D graphs	36
10.4	Fchk files	37
11	Integrals	39
11.1	One-body integrals	39
11.2	Two-body integrals	40
12	Hartree-Fock, HF	41
13	SCF	45
14	Potentials	47
15	Properties	49
16	Density Functional Theory, DFT	51
16.1	Subsystem embedding Options	52
17	Many-Body Perturbation Theory, MBPT	53
18	Propagator Theory, PT	55
19	Configuration Interaction, CI	57
19.1	CI Wavefunction	57
19.2	Selected CI, SCI	59
19.3	Natural orbitals	60
19.4	Diagonal corrections	60
19.5	CI Input options	60
20	Non-Orthogonal Configuration Interaction, NOCI	63
21	Tutorials	65
22	Positronic systems	67
23	Positron covalent bond	69
24	Quantum Nuclei	73
25	MP2 with quantum nucleus	75
26	PT2 with quantum nucleus	77
27	TOEP2 with quantum nucleus	79

28	PT3 for a Ps-atom complex	81
29	Negative Muons	83
30	Quantum Drude Oscillators, QDOs	85
31	External Potentials	87
32	Molden examples	89
32.1	Localized orbitals and fchk files	90
32.2	CI excited states	92
33	Cubes examples	95
33.1	Excited states	98
	Bibliography	101

openLOWDIN is a computational program that implements the Any Particle Molecular Orbital (APMO) method to study systems containing any type and number of quantum species, such as electrons, positrons, quantum nuclei, muons, or drude oscillators.

This manual is still in early construction stage! Thanks for your patience.

ABOUT

openLOWDIN is FORTRAN quantum chemistry code that implements the Any Particle Molecular Orbital (APMO) method to study systems containing any type and number of quantum species, such as electrons, positrons, quantum nuclei, muons, or drude oscillators. It's parallelized with OMP paradigm.

At present, openLOWDIN code is publicly available at <https://github.com/efposadac/openLOWDIN>

1.1 Capabilities

What can we do in Lowdin? The current version of the code encompasses the following quantum chemistry methods extended for any quantum species:

- HF
- DFT
- MP2
- CI (CIS, CISD, CIST, CISDTQ, FCI, CIPSI)
- PT (PT2, PT3, PP3, RENPP3, OVGf)
- NOCI

Check more details in the *Code* and *Tutorials*

1.2 Documentation

The online manual of openLOWDIN is available at https://github.com/openLOWDIN/openLOWDIN_manual. A compiled pdf version of the manual can be found here [openlowdin.pdf](#)

1.3 How to cite:

Please cite the code as:

R.i Flores-Moreno, E. Posada, F. Moncada, J. Romero, J. Charry, M. Díaz-Tinoco, S.A. González, N.F. Aguirre, A. Reyes, LOWDIN: The any particle molecular orbital code. Int. J. Quantum Chem.. 114. (1), 50–56 (2014).

1.4 History

openLOWDIN is the open-source version of LOWDIN code, which is the evolution of merging in 2010 two independent quantum chemistry codes and their projects. On one side, Andres Reyes' group developed the Any Particle Molecular Orbital code known as APMO, and Roberto Flores-Moreno led the development of the electronic structure package named PARAKATA (Butterfly in Purepecha, a native language of central-west Mexico). The resulting code has been named LOWDIN as a tribute to professor Per-Olov Löwdin for his dedication to create a global community in quantum chemistry, initiating many young Latin American scientists into this area. In the 1960, professor Löwdin founded the "International Winter Institutes" and their associated Sanibel Symposia, which were held annually at Sanibel Island, Florida, as an educational component to his renowned Summer Institutes in Uppsala, Sweden. These events provided intensive training for hundreds of scientists in the foundational principles of quantum chemistry, solid-state theory, and quantum biology, thereby significantly contributing to the international spread of quantum chemistry and inspiring generations of researchers, including the developers of this code.

1.5 Acknowledgements

This code results from an evolution of the APMO and LOWDIN software packages, both mainly developed at the Universidad Nacional de Colombia. Currently, the code is under an open-source initiative thanks to the support of some developers, who are presently spread around the world!

DEVELOPERS AND CONTRIBUTORS

2.1 Active developers (alphabetical order)

- Andrés Reyes
- Edwin Posada
- Jorge Charry
- Félix Moncada

2.2 Contributors (alphabetic order)

- Jhonathan Romero (APMO/Propagators)
- Sergio Gonzalez (APMO: Older versions)
- Nestor Aguirre (APMO: Older versions)
- Danilo González Forero (APMO/COSMO)
- Jose Mauricio Rodas Rodriguez (APMO/(QM/MM))
- Carlos Ortiz-Mahecha (APMO/CC)
- Alejandro Peña Torres (APMO/CC)
- Laura Pedraza-González (APMO/core)
- Manuel Diaz (APMO/Propagators)
- Teresa Tamayo-Mendoza (Development of higher order propagator methods)
- Roberto Flores-Moreno (ADFT/ADPT/Propagators)

INSTALLATION

3.1 Dependencies

openLOWDIN requires the following standard packages:

```
wget git build-essential liblapack-dev libblas-dev libgsl0-dev autotools-dev automake_
↪ libtool gfortran python3 gawk libeigen3-dev libgmp-dev libboost-all-dev
```

Additionally, it requires some quantum chemistry libraries:

```
libinit - Molecular integrals in gaussian type orbital basis
libxc - DFT functionals
```

The following opensource libraries are distributed within the openLOWDIN code

```
aduw - Four-index integrals transformation
erkale - Orbital localization
gepol - COSMO
jadamilu - large sparse matrix diagonalization
molden2aim - molden to AIM wave function converter
```

3.2 Getting the code

The source code is available at <https://github.com/efposadac/openLOWDIN>

```
git clone https://github.com/efposadac/openLOWDIN
```

3.3 Basic installation

Once all the dependencies are installed, the code is compile with the following steps. First, run the interactive configuration script in openLOWDIN root directory. Be sure that you have permissions to write in the installation directory and have properly exported the *\$PATH* environment.

```
./configure
```

This script will ask a set of questions, please provide the option that satisfies your needs.

```

INFO: Interactive configuration options
Fortran Compiler command? gfortran(default) or ifort/ix [gfortran]

Compiler Options: (1) regular, (2) backtrace and debug, (3) static (for intel fortran,
↳ compiler only), (4) Full debug, (5) Highest optimization level [1]

Speed up on GPUs? (you need to have already installed CUDA and Magma libraries): yes/no,
↳ [no]

Executable name? default=openlowdin [openlowdin]

Installation directory? default=/usr/local [/usr/local]

```

Compile the code with

```
make
```

you can add the parallel flag to compile in parallel, e.g. with 4 threads as `-j 4`.

Finally, install as

```
make install
```

To uninstall the binaries from the selected installation folder

```
make uninstall
```

To clean the project

```
make clean
make distclean
```

3.4 Step-by-step installation

Here you can find a step-by-step workflow to install on ubuntu-latest linux distribution.

```

### Step-by-step installation example: (replace apt-get with your preferred package
↳ manager) ###

sudo apt-get update
sudo apt-get -y install wget git build-essential liblapack-dev libblas-dev,
↳ libgsl0-dev autotools-dev automake libtool gfortran python3 gawk libeigen3-dev libgmp-
↳ dev libboost-all-dev
# Define ENV Variables
export WORKDIR=$PWD/dependencies
export PATH=$PATH:$WORKDIR/bin
export C_INCLUDE_PATH=$C_INCLUDE_PATH:$WORKDIR/include:$WORKDIR/include/libint2:/
↳ usr/include/eigen3
export CPLUS_INCLUDE_PATH=$CPLUS_INCLUDE_PATH:$WORKDIR/include:$WORKDIR/include/
↳ libint2:/usr/include/eigen3
export LIBRARY_PATH=$LIBRARY_PATH:$WORKDIR/lib
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$WORKDIR/lib

```

(continues on next page)

(continued from previous page)

```

# Create work directories
mkdir $WORKDIR
mkdir $WORKDIR/bin
mkdir $WORKDIR/lib
cd $WORKDIR

# Libint2
# If you have Ubuntu, you can get this precompiled Libint2 library
wget https://www.dropbox.com/s/d3d44j238lkfwr/libint-master-SEP052019.tgz
tar xzvf libint-master-SEP052019.tgz

# Otherwise, download and compile with minimal (default am), G12, fPIC options.
↪(libint2 commit 668b10c4bdca5876984058742d4212675eb93f3f)
# git clone https://github.com/evaleev/libint.git
# cd libint
# git checkout 668b10c4bdca5876984058742d4212675eb93f3f
# ./autogen.sh
# mkdir ../build
# cd ../build
# ../libint/configure --prefix=$WORKDIR --with-max-am=6 --enable-g12=4 --with-
↪g12-max-am=4 --with-cxxgen-optflags
# make -j 4
# make install
# ../libint/configure --prefix=$WORKDIR

cd -

# Libint1
git clone https://github.com/evaleev/libint.git
cd libint
git checkout v1
aclocal -I lib/autoconf
autoconf
./configure --prefix=$WORKDIR
make -j 4
make install
make clean
make distclean
cd -

# Libxc
cd $WORKDIR
# If you have Ubuntu, you can get this precompiled Libxc library
wget https://www.dropbox.com/s/6cja3zzhl1cq46i/libxc-master-MAY242023.tgz
tar xzvf libxc-master-MAY242023.tgz
# Otherwise, download and compile with default options (libxc commit.
↪4bd0e1e36347c6d0a4e378a2c8d891ae43f8c951)
# git clone https://gitlab.com/libxc/libxc.git
# cd libxc
# git checkout 4bd0e1e36347c6d0a4e378a2c8d891ae43f8c951
# autoreconf -i
# ./configure --enable-shared --prefix=$WORKDIR

```

(continues on next page)

(continued from previous page)

```
# make -j 4
# make install

cd ..

# Configure Lowdin
./configure -p $WORKDIR/bin -s /tmp -l "-lblas -llapack"
# Build Lowdin
make -j 4
# Install Lowdin
make install
# Run Tests
make test
```


GETTING STARTED

Let's get ready to run openLOWDIN. Here you can find the basic information about the input and how to run the code. A more exhaustive description of all code keywords and files can be found in section [Code](#)

4.1 Input file

The code requires a plain text input file with extension `.lowdin`. Here is an example of a minimum input file for computing muonic water with propagator theory

```

GEOMETRY
e-[O]      6-31G      0.0000      0.0000      0.1173 multiplicity=2
↪addParticles=-1
e-[H]      6-31G      0.0000      0.7572     -0.4692
e-[H]      6-31G      0.0000     -0.7572     -0.4692
u-      13S.ET.O.u.TF  0.0000      0.0000      0.1173
0      dirac      0.0000      0.0000      0.1173
H-a_1      DZSPDN      0.0000      0.7572     -0.4692 m = 1836.15
H-b_2      DZSPDn      0.0000     -0.7572      0.4692 m = 1836.15
END GEOMETRY

TASKS
      method = "UHF"
      propagatorTheoryCorrection = 2
END TASKS

CONTROL
      iterationScheme = 3
      convergenceMethod = 1
      readCoefficients = F
      hartreeProductGuess = T
END CONTROL

```

The minimum required blocks to run a calculation are GEOMETRY, TASKS, and CONTROL.

The GEOMETRY block provides the information needed to build the molecular system. The first column declares the type of the quantum species. As shown in the above example, e-[H] and e-[O] define the electrons of a Hydrogen and a Oxygen atom respectively; U- defines a negative muon, 0 dirac, H_1 and H_2 define a ^{16}O , ^1H and ^2H nuclei respectively.

The second column declares the basis sets. When the dirac basis is chosen, the particle is treated as a classical point charge. The third, fourth and fifth columns declare the x, y, z coordinates of the particle basis set center.

The sixth column provides additional information via keywords `addParticles` and `multiplicity`. These keywords are used to change the default values. `addParticles` is used to modify the number of particles of a quantum species. As shown in the provided example, one electron is removed from the system. `multiplicity` defines the multiplicity for open shell calculations. In the example, an electronic multiplicity of 2 was chosen.

4.2 How to run

To run openLOWDIN with 4 OMP threads simply run

```
openlowdin -i inputname.lowdin -n 4
```

This will generate a plain text output file called `inputname.out`. These are the full command line options:

```
$ openlowdin -i file.lowdin [-t [all] [list] [file]] [-n number] [-v number] [-p] [-s] [-h]
-i file.lowdin
    This is the input file name
-n number
    This will set the number of OMP threads
-t all
    This will run all the test files located on the test database.
-t list
    This will list all the test files located on the test database.
-t file
    This will run a specific test file which is located on test database.
-v number
    This is the lowdin version that will be used
-p
    This will print the output file to the standard output on the fly
-w
    This will save the LOWDIN .wfn file
-k
    This will keep the temporary files in the scratch directory after running the calculation
-s
    This activate the singleton mode
-h
    This will print this same message
```

4.3 Scratch

openLOWDIN creates a folder to save temporary files located at `$LOWDIN_SCRATCH/$nameFile` where `$nameFile` is the input name given after the command line `-i file.lowdin` without the `.lowdin` extension. The `$LOWDIN_SCRATCH` environmental variable is set in the bash script `.openlowdin/lowdinvars.sh` which is located in the installation folder during configuration.

Warning: Please notice that the scratch folder is completely removed after the calculation terminates.

PUBLICATIONS

Here is the full list of publications using openLOWDIN code.

5.1 Papers

1. L. Martins et al. ‘Positronium Atoms Solvated in Liquid Alcohols: A Multicomponent Quantum Mechanics/Molecular Mechanics Approach’. *J. Phys. Chem. B* **129**, 12503 (2025) [10.1021/acs.jpcc.5c06150](https://doi.org/10.1021/acs.jpcc.5c06150).
2. F. Moncada, A. Reyes, and L. G. M. Pettersson. ‘Restoring rotational symmetry of multicomponent wavefunctions with nuclear orbitals’. *J. Chem. Phys.* **162**, 024110 (2025) [10.1063/5.0244318](https://doi.org/10.1063/5.0244318).
3. F. Moncada, W. Quintero, E. Posada, L. G. M. Pettersson, and A. Reyes. ‘A nuclear configuration interaction approach to study nuclear spin effects: an application to ortho- and para- $3\text{He}2@\text{C}60$ ’. *ChemPhysChem* **25**, e202300498 (2024) [10.1002/cphc.202300498](https://doi.org/10.1002/cphc.202300498).
4. D. Archila-peña et al. ‘Two-Positron-bonded Dihalides : $\text{Ps}2\text{XY}$ (X, Y = F, Cl, Br)’. *Chem. Eur. J.* **30**, 202402618 (2024) [10.1002/chem.202402618](https://doi.org/10.1002/chem.202402618).
5. M. Bergami, J. Charry, A. Reyes, K. Coutinho, and M. T. D. N. Varella. ‘Does Positron Attachment Take Place in Water Solution?’ *J. Phys. Chem. B* **128**, 10178 (2024) [10.1021/acs.jpcc.4c03627](https://doi.org/10.1021/acs.jpcc.4c03627).
6. R. Porras-Roldan et al. ‘Electron superhalogens as positronium superhalogens’. *Phys. Chem. Chem. Phys.* **26**, 18881 (2024) [10.1039/d4cp01221f](https://doi.org/10.1039/d4cp01221f).
7. F. Moncada and A. Reyes. ‘Multicomponent wavefunction-in-DFT embedding for positronium molecules’. *J. Chem. Phys.* **158**, 134101 (2023) [10.1063/5.0139813](https://doi.org/10.1063/5.0139813).
8. M. Bergami et al. ‘Multicomponent Quantum Mechanics/Molecular Mechanics Study of Hydrated Positronium’. *J. Phys. Chem. B* **126**, 2699 (2022) [10.1021/acs.jpcc.1c10124](https://doi.org/10.1021/acs.jpcc.1c10124).
9. J. Charry et al. ‘The three-center two-positron bond’. *Chemical Science* **13**, 13795 (2022) [10.1039/d2sc04630j](https://doi.org/10.1039/d2sc04630j).
10. F. Moncada, L. Pedraza-González, J. Charry, M. T. do N. Varella, and A. Reyes. ‘Covalent bonds in positron dihalides’. *Chem. Sci.* **11**, 44 (2020) [10.1039/C9SC04433G](https://doi.org/10.1039/C9SC04433G).
11. J. Rodas, J. F. Galindo, A. E. Roitberg, and A. Reyes. ‘The any particle molecular orbital/molecular mechanics approach’. *J. Mol. Model.* **25**, 316 (2019) [10.1007/s00894-019-4153-x](https://doi.org/10.1007/s00894-019-4153-x).
12. A. Reyes, F. Moncada, and J. Charry. ‘The any particle molecular orbital approach: A short review of the theory and applications’. *Int. J. Quantum Chem.* **119**, 1 (2019) [10.1002/qua.25705](https://doi.org/10.1002/qua.25705).
13. J. Charry, M. T. do N. Varella, and A. Reyes. ‘Binding Matter with Antimatter: The Covalent Positron Bond’. *Angew. Chem.* **57**, 8859 (2018) [10.1002/anie.201800914](https://doi.org/10.1002/anie.201800914).
14. F. Moncada, R. Flores-Moreno, A. Reyes. ‘Theoretical calculation of polarizability isotope effects’ *J. Mol. Model.* **27**, 90 (2017). [10.1007/s00894-017-3236-9](https://doi.org/10.1007/s00894-017-3236-9).

15. L. Pedraza-González, J. A. Charry Martínez, W. D. Quintero, J. Ali-Torres, and A. Reyes. ‘Fast and Accurate Prediction of Proton Affinities: Revisiting the Extended Koopmans’ Theorem for Protons’. *Phys. Chem. Chem. Phys.* **19**, 25324 (2017) [10.1039/C7CP04936F](#).
16. J. Charry, L. Pedraza-González, and A. Reyes. ‘On the physical interpretation of the nuclear molecular orbital energy’. *J. Chem. Phys.* **146**, 214103 (2017) [10.1063/1.4984098](#).
17. L. Pedraza-González, J. Romero, J. Alí-Torres, and A. Reyes. ‘Prediction of proton affinities of organic molecules using the any-particle molecular-orbital second-order proton propagator approach’. *Phys. Chem. Chem. Phys.* **18**, 27185 (2016) [10.1039/C6CP05128F](#).
18. E. Posada, F. Moncada, and A. Reyes. ‘Negative Muon Chemistry: The Quantum Muon Effect and the Finite Nuclear Mass Effect’. *J. Phys. Chem. A* **118**, 9491 (2014) [10.1021/jp501289s](#).
19. J. Charry, J. Romero, M. T. do N. Varella, and A. Reyes. ‘Calculation of positron binding energies of amino acids with the any-particle molecular-orbital approach’. *Phys. Rev. A* **89**, 052709 (2014) [10.1103/PhysRevA.89.052709](#).
20. R. Flores-Moreno et al. ‘LOWDIN: The any particle molecular orbital code’. *Int. J. Quantum Chem.* **114**, 50 (2014) [10.1002/qua.24500](#).
21. J. Romero, J. A. Charry, R. Flores-Moreno, M. T. D. N. Varella, and A. Reyes. ‘Calculation of positron binding energies using the generalized any particle propagator theory.’ *J. Chem. Phys.* **141**, 114103 (2014) [10.1063/1.4895043](#).
22. M. Díaz-Tinoco et al. ‘A generalized any-particle propagator theory: Prediction of proton affinities and acidity properties with the proton propagator’. *J. Chem. Phys.* **138**, 194108 (2013) [10.1063/1.4805030](#).
23. F. Moncada, D. Cruz, and A. Reyes. ‘Electronic properties of atoms and molecules containing one and two negative muons’. *Chem. Phys. Lett.* **570**, 16 (2013) [10.1016/j.cplett.2013.03.004](#).
24. J. Romero, E. Posada, R. Flores-Moreno, and A. A. A. A. Reyes. ‘A generalized any particle propagator theory: Assessment of nuclear quantum effects on electron propagator calculations’. *J. Chem. Phys.* **137**, 74105 (2012) [10.1063/1.4745076](#).
25. F. Moncada, D. Cruz, and A. Reyes. ‘Muonic alchemy: Transmuting elements with the inclusion of negative muons’. *Chem. Phys. Lett.* **539–540**, 209 (2012) [10.1016/j.cplett.2012.04.062](#).
26. D. Moreno, S. a González, and A. Reyes. ‘Turning symmetric an asymmetric hydrogen bond with the inclusion of nuclear quantum effects: the case of the [CN $\cdots$ H $\cdots$ NC] $^-$ complex.’ *J. Chem. Phys.* **134**, 024115 (2011) [10.1063/1.3521272](#).
27. S. González, N. Aguirre, and A. Reyes. ‘Theoretical Investigation of Isotope Effects: The Any-Particle Molecular Orbital Code’. *Int. J. Quantum Chem.* **108**, 1742 (2008) [10.1002/qua.21584](#).
28. S. González, N. F. Aguirre, and A. Reyes. ‘Apmo: Un Programa Computacional Para El Estudio De Efectos Cuánticos Nucleares Mediante La Teoría Del Orbital Molecular Electrónico Y No Electrónico’. *Rev. Colomb. Quím.* **37**, 93 (2008).

5.2 Thesis

List of Master and PhD thesis:

1. Rafael Porras Roldan. [Development of MP2 orbital optimization and spin scaling methods for positron-molecule systems](#). Master Thesis. Supervisor Andrés Reyes Velasco. 2025
2. Leonardo Bin Martins. [Átomos de positrônio em meios condensados](#). PhD thesis. Supervisor Marcio Teixeira do Nascimento Varella. 2025

3. Andre Luis Dias Santana [Moléculas positrônicas neutras](#). PhD thesis. Supervisor Marcio Teixeira do Nascimento Varella. 2023
4. Jorge Alfonso Charry Martinez. [Chemistry and physics of positrons interacting with atoms, molecules, and fields](#). PhD thesis. Supervisor Alexandre Tkatchenko. 2023
5. Félix Santiago Moncada Arias. [Development of a multicomponent wavefunction-in-DFT embedding methodology](#) PhD thesis. Supervisor Andrés Reyes Velasco. 2021
6. Leonardo Bin Martins. [Annihilation rates in positronic molecules](#). Master thesis. Supervisor Marcio Teixeira do Nascimento Varella. 2020
7. William David Quintero Martinez. [Estudio teórico del confinamiento de gases nobles en fullerenos](#). Master thesis. Supervisor Andrés Reyes Velasco. 2018
8. Edwin Fernando Posada Correa. [Desarrollo e implementación de un método libre de base para el estudio de efectos cuánticos nucleares \(Nu-APMO: Numerical Any Particle Molecular Orbital\)](#). PhD thesis Supervisor Andrés Reyes Velasco. 2017
9. Andre Luis Dias Santana. [Moléculas positrônicas: densidade eletrônica e taxas de aniquilação](#). Master thesis. Supervisor Marcio Teixeira do Nascimento Varella. 2017
10. Carlos Andrés Ortiz-Mahecha. [Extensión teórica e implementación computacional del método CCSD\(T\) en el marco de la teoría APMO](#). Master thesis. Supervisor Andrés Reyes Velasco. 2017
11. Danilo González Forero. Master thesis. Supervisor Andrés Reyes Velasco. 2016
12. Laura Milena Pedraza González. [Análisis de la unión de positrones a moléculas empleando el método del orbital molecular para cualquier tipo de partícula \(APMO\)](#). Master thesis. Supervisor Andrés Reyes Velasco. 2016
13. Matheus Rodríguez Álvarez. [Extensión de la descomposición de energía para el análisis de mecanismos de reacción dentro de la teoría de fuerza y flujo electrónico de reacción](#). Master thesis. Supervisor Andrés Reyes Velasco. 2016
14. Alejandro Peña Torres. [Desarrollo de métodos de Coupled Cluster en el marco de la teoría de orbital molecular para cualquier partícula \(APMO\) y su implementación en LOWDIN](#). Master thesis. Supervisor Andrés Reyes Velasco. 2015
15. Jorge Alfonso Charry Martínez. [Desarrollo e implementación de un método de funciones gaussianas explícitamente correlacionadas bajo el esquema de orbitales moleculares para cualquier partícula, APMO](#). Master thesis. Supervisor Andrés Reyes Velasco. 2015
16. Ronald González Medina. [Implementación de un método de orbitales moleculares de fragmentos para cualquier partícula en el paquete computacional LOWDIN](#) Master thesis. Supervisor Andrés Reyes Velasco. 2014
17. Félix Santiago Moncada Arias. [Desarrollo teórico e implementación computacional de una metodología no Born-Oppenheimer de teoría de funcionales de la densidad basada en el método del orbital molecular nuclear con escalamiento cúbico para el estudio de efectos cuánticos nucleares](#) Master thesis. Supervisor Andrés Reyes Velasco. 2013
18. Jhonathan Romero Fontalvo. [Desarrollo de métodos de propagador para el estudio de procesos de ionización molecular empleando la metodología del orbital nuclear molecular y su implementación en el paquete computacional LOWDIN](#). Master thesis. Supervisor Andrés Reyes Velasco. 2013
19. Edwin Fernando Posada Correa. [Optimización del paquete computacional para el cálculo de estructura núcleo-electrónica \(APMO\)](#). Master thesis. Supervisor Andrés Reyes Velasco. 2011
20. Nestor Aguirre. Master thesis. Supervisor Andrés Reyes Velasco. 2010
21. Sergio Arquimedes. González Mónico. [Estudio teórico de los efectos de isótopo en puentes de hidrógeno y su aplicación a agregados de agua mediante metodología no Born-Oppenheimer](#) Master thesis. Supervisor Andrés Reyes Velasco. 2010

CODE

This sections summarizes the multicomponent version of the common quantum chemistry methods and capabilities implemented in openLOWDIN, as well as of all available input code keywords.

- *Input*
- *CONTROL*
- *Lib*
- *Outputs*
- *Integrals*
- *Hartree-Fock, HF*
- *SCF*
- *Potentials*
- *Properties*
- *Density Functional Theory, DFT*
- *Many-Body Perturbation Theory, MBPT*
- *Propagathor Theory, PT*
- *Configuration Interaction, CI*
- *Non-Orthogonal Configuration Interaction, NOCI*

7.1 General structure

openLOWDIN input file consists on a plain text file with extension `.lowdin` which will be internally processed by a bash script to generate a Fortran namelist, creating the true input file for the code. The input file consists of different blocks enclosed by the lines `BLOCK` and `END BLOCK`, where “BLOCK” is one of the following sections:

- **GEOMETRY**: define the position of the basis set centers, position of point charges, number and properties of quantum species.
- **TASKS** : select the type of calculation to be performed by the code.
- **CONTROL**: contains all general input parameters to control the behavior of the program, such as thresholds, maximum number of SCF cycles. See [CONTROL](#) for a full of keywords.
- **OUTPUT**: request the calculation of other molecular property to visualize the molecular wave function or density. See [Outputs](#)
- **BASIS** : declare a user defined basis. This can be alternatively defined in an external file. See [Basis](#)
- **INTERPOTENTIAL** define an internal potential between pairs of quantum species. See [Potentials](#)
- **EXTERPOTENTIAL** define an external potential for a quantum species. See [Potentials](#)
- **POTENTIAL** declare a user defined potential built on gaussian type orbitals. See [Potential Basis](#)
- **INPUT_CI** define the active space for each quantum species for CI calculations. See [Configuration Interaction, CI](#)

7.2 GEOMETRY block

The GEOMETRY block provides the information needed to build the molecular system as:

```
GEOMETRY
SPECIES_SYMBOL    BASIS_NAME  X_COORD Y_COORD Z_COORD
END GEOMETRY.
```

each line define either a new particle (quantum or point charge), or a new basis center (quantum particles only).

- **SPECIES_SYMBOL** [*character*] symbol of the quantum particle as defined in the `lib/dataBases/elementalParticles.lib` and `lib/dataBases/constantsOfCoupling.lib` files, or the symbol of point charges defined in `lib/dataBases/atomicElements.lib`. See [DataBases](#) to define or redefine particles.
- **BASIS_NAME** [*character*] for quantum particles, this corresponds to the name of the basis set, the code will look in the `BASIS` block or in the folder `lib/basis`. For point charge simply use `dirac``

- X_COORD, Y_COORD, Z_COORD [*real*] coordinates of the basis set center origin or position of the point charge.

Besides the above mandatory elements, the following flags can be added to modify the properties of the particles declared within the same line.

- addParticles= [*integer*] modify the number of particles of a quantum species. Positive values increase number of particles, negative values decrease the number of particles. To not be confused with total charge. *Default* 0
- multiplicity= [*integer*] declare the multiplicity of the quantum species as $2S + 1$, where S is the total spin within the quantum species. *Default* 1
- q= [*real*] charge of the particle, quantum or point charge. *Default* defined in lib/dataBases/elementalParticles.lib
- m= [*real*] mass of the particle, quantum only. *Default* defined in lib/dataBases/elementalParticles.lib
- omega= [*real*] frequency of the harmonic potential of the particle, quantum only. *Default* 0.0 See [Potentials](#)
- qdoCenterOf= [*character*] declare this particle as the point charge center of the quantum species in [*character*] within the QDO formalism. *Default* "NONE" See [Quantum Drude Oscillators, QDOs](#)

Note that a duplicate of one of the above keywords will overwrite the previous values for particles defined with the same name.

7.3 TASKS block

This block control the method to be computed

- method= [*character*] Type of Hartree-Fock method See [Hartree-Fock, HF](#)

"RHF"	Restricted Hartree-Fock
"UHF"	Unrestricted Hartree-Fock
"NONE"	No Hartree-Fock. <i>Default</i>

- optimizeGeometry= [*logical*] Activates geometry optimization. *Default* False
- mollerPlessetCorrection= [*integer*] Order of Möller-Plesset Perturbation correction. Values: 2. *Default* 0 See [Many-Body Perturbation Theory, MBPT](#)
- epsteinNesbetCorrection= [*integer*] Order of Epstein-Nesbet correction. Values: 2, 3. *Default* 0 See [Many-Body Perturbation Theory, MBPT](#)
- propagatorTheoryCorrection= [*integer*] Order of propagator theory correction. Values: 2, 3. *Default* 0 See [Propagator Theory, PT](#)
- nonOrthogonalConfigurationInteraction= [*logical*] Performs non-Orthogonal Configuration Interaction calculation. *Default* False See [Non-Orthogonal Configuration Interaction, NOCI](#)
- configurationInteractionLevel= [*character*] Select Configuration Interaction level. See [Configuration Interaction, CI](#)

"CIS"	Singles
"CISD"	Singles and doubles
"CISD-"	Singles, doubles interspecies only
"CISD+"	Singles, doubles, triples interpecies only
"CISDT"	Singles, doubles, triples
"CISDTQ"	Singles, doubles, triples, quadruples
"SCI"	Selected CI
"FCI"	Full CI
"NONE"	<i>Default</i>

- TDHF= *[logical]* *Default False*
- cosmo= *[logical]* Performs an implicit solvent job using COSMO model. *Default False*
- subsystemEmbedding= *[logical]* *Default False*

openLOWDIN employs external files to obtain information regarding particles properties, basis sets, and potential basis. Those files are located in the folder openLOWDIN/lib under the following tree structure: .. code

```
|— basis
|— dataBases
|— libjadamilu.a
|— libmolden2aim.a
|— libtransform.a
|— lowdincore.a
|— potentials
```

Notice that all *.a files corresponds to static external libraries compiled during openLOWDIN installation.

8.1 DataBases

Information regarding particle properties are stored in three different files, while uffParameters.lib contains the Universal Force Field (UFF) parameters

```
|— atomicElements.lib
|— constantsOfCoupling.lib
|— elementalParticles.lib
|— uffParameters.lib
```

8.1.1 atomicElements.lib

This file stores some physicochemical for all elements in the periodic table in a Fortran namelist format. This information is mostly used to get the charge of classical particles and number of electrons from the given atomic element, as well as the mass of quantum nuclei

```
&ELEMENT
  NAME = "HYDROGEN"
  SYMBOL = "H"
  ATOMICNUMBER = 1
  MASS = 1.00794
  MELTINGPOINT = 13.81
  BOILINGPOINT = 20.28
  DENSITY = 0.084
  ELECTRONAFFINITY = -73
```

(continues on next page)

(continued from previous page)

```

IONIZATIONENERGY1 = 1312
ELECTRONEGATIVITY = 2.1
COVALENT = 0.30
ATOMIC = 0.25
KLAMT = 0.00
VANDERWAALS = 1.2
ISOTOPES = 1, 1.0078250321, 99.9885, 0.5,
           2, 2.0141017780, 0.0115, 1,
           3, 3.0160492675, 0.0, 0.5,
           4, 4.02783, , -2.0,
           5, 5.03954, , ,
           6, 6.04494, , ,
           7, 3.1289311806, , 1,
/

```

8.1.2 constantsOfCoupling.lib

lambda, eta, and particlesFraction are related to the occupation of quantum species (number of particles) per orbital, this variable is used in post-HF methods and nuclear gradients. kappa is used to change the sign of the exchange integrals. See *Hartree-Fock, HF*

```

&SPECIE
NAME = "ELECTRON"
SYMBOL = "E-"
KAPPA = -1.0
ETA = 2.0
LAMBDA = 2.0
PARTICLESFRACTION = 0.5
/

```

8.1.3 elementalParticles.lib

this namelist defines the elemental properties of quantum particles: charge, mass, and spin

```

&PARTICLE
NAME = "ELECTRON"
SYMBOL = "E-"
CATEGORY = "LEPTON"
CHARGE = -1
MASS = 1.0
SPIN = 0.5
/

```

Note: To create a new unique (distinguishable) quantum species is necessary to add a new entry into both namelist on constantsOfCoupling.lib and elementalParticles.lib which an unique SYMBOL not used by another other quantum species

Note: openLOWDIN has three exceptions for quantum species. Electrons, defined by "E-", in which the number

of particles and basis set are defined by adding the atomic symbol in parenthesis, e.g. E-(N) will declare to add 7 electrons to the quantum species "E-" and to add a set of Gaussian type functions identified by the name "NITROGEN" from the basis set requested. The quantum species E-ALPHA and E-BETA are automatically created when requesting and UHF calculation.

Note: The mass of quantum nuclei can be selected by adding to the species symbol the isotope number followed by “_” e.g. H_3 will define a Tritium nucleus.

8.2 Basis

openLOWDIN constructs the spatial part of spin-molecular orbitals, χ as a linear combination of Gaussian type functions (GTFs), ϕ_μ^α :

$$\phi_i^\alpha(\mathbf{r}_i) = \sum_{\mu}^{N_{bas}^\alpha} C_\mu^\alpha \varphi_\mu^\alpha(\mathbf{r}_i; \mathbf{R}_\mu) \quad (8.1)$$

where C_μ^α is a combination coefficient for species α , and the atomic orbital $\varphi_\mu^\alpha(\mathbf{r}_i; \mathbf{R}_\mu)$ is built as a sum of primitive functions forming a contracted orbital

$$\varphi_\mu^\alpha(\mathbf{r}_i; \mathbf{R}_\mu) = \sum_s b_{s\mu}^\alpha N_{s\mu}^\alpha (x_i - X_\mu)^{l_\mu^\alpha} (y_i - Y_\mu)^{m_\mu^\alpha} (z_i - Z_\mu)^{n_\mu^\alpha} \times \exp[-a_s^\alpha (\mathbf{r}_i - \mathbf{R}_\mu)^2] \quad (8.2)$$

here, $b_{s\mu}^\alpha$ are the primitive coefficients, $N_{s\mu}^\alpha$ is a normalization constant, $\{x_i, y_i, z_i\}$ are the spatial coordinates of the position vector $\mathbf{r}$ for the i particle of the quantum species α , the set $\{l, m, n\}$ are the angular momentum components, a_s is the exponent, and $\mathbf{R}$ is the GTF center, which is independent of nuclei positions.

openLOWDIN has a collection of built-in basis stored in `lib/basis/`. Please note that all filenames should be written in capital letters. The code employs the same basis set format of deMon2k, That is to say

```
O-ELEMENT_NAME ELEMENT_SYMBOL_OR_SPECIES_SYMBOL (BASIS_NAME) BASIS TYPE: 1
#
NUMBER_OF_CONTRACTED_ORBITALS
ID ANGULAR_MOMENTUM NUMBER_OF_PRIMITIVES
EXPONENT COEFFICIENT
```

For example, for a Hydrogen atom center with aug-cc-pVDZ basis

```
O-HYDROGEN H (AUG-CC-PVDZ) BASIS TYPE: 1
#
5
1 0 3
13.01000000 0.01968500
1.96200000 0.13797700
0.44460000 0.47814800
2 0 1
0.12200000 1.00000000
3 0 1
0.02974000 1.00000000
4 1 1
0.72700000 1.00000000
5 1 1
0.14100000 1.00000000
```

Alternatively, it's possible to define the basis set of a quantum species within the `.lowdin` input file by creating a BASIS BASIS_NAME - END BASIS block section, where BASIS_NAME should be a basis name defined within the GEOMETRY block. For example

```

GEOMETRY
  e-[H]  cc-pvtz      0.0000  0.0000  0.00000
  e-[H]  CUSTOM_1    0.0000  0.0000  0.74144
  H_1    CUSTOM_2    0.0000  0.0000  0.00000 m=1836.1527
  U-     CUSTOM_3    0.0000  0.0000  0.74144 m=206.7683
  He_4   CUSTOM_3    0.0000  0.0000  0.74144 m=7349.6727
END GEOMETRY

BASIS CUSTOM_1
O-HYDROGEN H (CC-PVTZ+LOCAL) BASIS TYPE: 1
#
9
1 0 1
103.8700000 1.00000000
2 0 1
33.8700000 1.00000000
3 0 1
5.09500000 1.00000000
4 0 1
1.15900000 1.00000000
5 0 1
0.32580000 1.00000000
6 0 1
0.10270000 1.00000000
7 1 1
1.40700000 1.00000000
8 1 1
0.38800000 1.00000000
9 2 1
1.05700000 1.00000000
END BASIS

```

If the basis set is not provided within the input file, then the code will look in the `lib/basis` folder for a basis set file under the name defined in the GEOMETRY block, e.g. a file `lib/basis/CUSTOM_1` in the above example.

A more extensive collection of updated basis set can be found <https://www.basissetexchange.org/> or https://github.com/MolSSI-BSE/basis_set_exchange

8.3 Potential Basis

openLOWDIN has the capabilities of computing additional one-body external potential, as well as replacing the standard two-particles Coulomb potential by a general two-body intraspecies and interspecies potentials based on sum Gaussian-Type functions (GTFs).

8.3.1 External potential basis

This potential is built as a sum of uncontracted and unnormalized GTFs

$$V_1^\alpha(\mathbf{r}_i) = \sum_{\tau}^{N_{bas}^\alpha} C_\tau^\alpha (x_i - X_\tau)^{l_\tau^\alpha} (y_i - Y_\tau)^{m_\tau^\alpha} (z_i - Z_\tau)^{n_\tau^\alpha} \times \exp[-a_\tau^\alpha (\mathbf{r}_i - \mathbf{R}_\tau)^2] \quad (8.3)$$

where all parameters are defined in a similar fashion that the one defined for basis sets. The format of this potential basis corresponds to

```
O-SPECIES_SYMBOL
#
NUMBER_OF_FUNCTIONS
ID  ANGULAR_MOMENTUM
EXPONENT  COEFFICIENT
ORIGIN_X ORIGIN_Y ORIGIN_Z
```

For example, for a 4*s* potential felt by the species HEA3

```
O-HEA3
#
25
1 0
0.00100000 -4.85267703e-04
0.0 0.0 0.0
2 0
0.00200000 2.44420303e-03
0.0 0.0 0.0
3 0
0.00400000 -7.54493346e-03
0.0 0.0 0.0
4 0
0.00800000 1.57739046e-02
0.0 0.0 0.0
```

In the input file, this potential is invoked by adding a new line in EXTERPOTENTIAL - END EXTERPOTENTIAL block section as

```
EXTERPOTENTIAL
    SPECIES_SYMBOL  POT_NAME
END EXTERPOTENTIAL
```

where POT_NAME is the name of the potential file described above for the given species SPECIES_SYMBOL. For example

```
EXTERPOTENTIAL
    HEA3    HE2C60-IH-1P
```

(continues on next page)

(continued from previous page)

```

      HEB3   HE2C60-IH-1P
END  EXTERPOTENTIAL

```

Note: Notice that it's possible to define a potential for different quantum species within the same file.

8.3.2 Internal potential basis

This potential is also built as a sum of uncontracted and unnormalized geminal GTFs, but it is limited to *s*-type orbitals

$$V_2^{\alpha\beta}(\mathbf{r}_i^\alpha, \mathbf{r}_j^\beta) = \sum_{\tau}^{N_{bas}^{\alpha\beta}} C_{\tau}^{\alpha\beta} \exp[-a_{\tau}^{\alpha\beta}(\mathbf{r}_i^\alpha - \mathbf{r}_j^\beta)^2] \quad (8.4)$$

And the potential format is given by

```

O-SPECIES_SYMBOL_ALPHA SPECIES_SYMBOL_BETA
#
NUMBER_OF_FUNCTIONS
ID  ANGULAR_MOMENTUM
EXPONENT  COEFFICIENT
ORIGIN_X ORIGIN_Y ORIGIN_Z

```

For a 4*s* potential between two quantum species HEA3 and HEB3

```

O-HEA3HEB3
4
0 0
      10.00000000000000      16.533492358000
0.0 0.0 0.0
1 0
      7.498942093300      -6.924850494800
0.0 0.0 0.0
2 0
      5.623413251900      5.780268985500
0.0 0.0 0.0
3 0
      4.216965034300      1.957403910100
0.0 0.0 0.0
4 0
      3.162277660200      -5.166130478700

```

In the input file, this potential is invoked by adding a new line in INTERPOTENTIAL - END INTERPOTENTIAL block section as

```

INTERPOTENTIAL
      SPECIES_SYMBOL_ALPHA SPECIES_SYMBOL_BETA  POT_NAME
END  INTERPOTENTIAL

```

where POT_NAME is the name of the potential file described above between the species SPECIES_SYMBOL_ALPHA and SPECIES_SYMBOL_BETA. For example

```
INTERPOTENTIAL
    HEA3 HEA3    HE2C60-IH-2P
    HEA3 HEB3    HE2C60-IH-2P
    HEB3 HEB3    HE2C60-IH-2P
END INTERPOTENTIAL
```

Note: Notice that it's possible to define a potential for different quantum species within the same file.

Note: Notice that interspecies potentials can be simply declared by setting $\alpha = \beta$, in other words, by setting the same quantum species symbol twice.

Warning: For simplicity, openLOWDIN requires to read an angular momentum and origin for the two-body potentials, despite these are not used in the potential definition.

CONTROL

9.1 Dummy variables

For debugging purposes

- `dummyReal(:)=[float]` Dummy real array(10). *Default 0.0_8*
- `dummyInteger(:)=[integer]` Dummy integer array(10). *Default 0*
- `dummyLogical(:)=[logical]` Dummy logical array(10). *Default .false.*
- `dummyCharacter(:)=[character]` Dummy character array(10). *Default ""*

9.2 Geometry optimization

Currently unsupported

- `numericalDerivativeDelta=[float]` *Default 1.0E-3*
- `minimizationInitialStepSize=[float]` *Default 0.5_8*
- `minimizationLineTolerance=[float]` *Default 0.001_8*
- `minimizationToleranceGradient=[float]` *Default 0.00001_8*
- `minimizationMaxIteration=[integer]` *Default 200*
- `minimizationMethod=[integer]` *Default 4*
- `minimizationLibrary=[character]` *Default "GENERIC"*
- `coordinates=[character]` *Default "CARTESIAN"*
- `energyCalculator=[character]` *Default "INTERNAL"*
- `analyticGradient=[logical]` *Default .true.*
- `minimizationWithSinglePoint=[logical]` *Default .true.*
- `useSymmetryInMatrices=[logical]` *Default .false.*
- `restartOptimization=[logical]` *Default .false.*
- `firstStep=[logical]` *Default .true.*
- `lastStep=[logical]` *Default .true.*
- `optimizeWithCpCorrection=[logical]` *Default .false.*
- `cpCorrection=[logical]` *Default .false.*

- TDHF= *[logical] Default .false.*
- optimize= *[logical] Default .false.*
- optimizeGeometryWithMP= *[logical] Default .false.*
- projectHessiane= *[logical] Default .true.*

9.3 Atomic connectivity

Currently unsupported

- bondDistanceFactor= *[float] Default 1.3_8*
- bondAngleThreshold= *[float] Default 170.0_8*
- dihedralAngleThreshold= *[float] Default 170.0_8*

9.4 COSMO

Currently unsupported

- cosmo= *[logical] Default .false.*
- cosmo_solvent_dielectric= *[character] Default 78.3553d+00*
- cosmo_scaling= *[character] Default 0.0d+00*

9.5 Info and units

- formatNumberOfColumns= *[integer] Default 5*
- unitForOutputFile= *[integer] Default 6*
- unitForMolecularOrbitalsFile= *[integer] Default 8*
- unitForMP2IntegralsFile= *[integer] Default 7*
- printLevel= *[integer]*

0	No output
1	Normal. <i>Default</i>
5	Method
6	Method and wave function
7	Method, wave function, global
8	Method, wave function, global, SCF

- units= *[character] Default "ANGS"*
- doubleZeroThreshold= *[float] Default 1.0E-12*

9.6 General

- `method= [character] Default "NONE"`
- `transformToCenterOfMass= [logical] Default .false.`
- `areThereDummyAtoms= [logical] Default .false.`
- `areThereQDOPotentials= [logical] Default .false.`
- `setQDOEnergyZero= [logical] Default .false.`
- `isThereExternalPotential= [logical] Default .false.`
- `isThereInterparticlePotential= [logical] Default .false.`
- `isThereOutput= [logical] Default .false.`
- `isThereFrozenParticle= [logical] Default .false.`
- `dimensionality= [integer] Default 3`

9.7 Molecular Mechanics

Currently unsupported

- `forceField= [character] Default "UFF"`
- `electrostaticMM= [logical] Default .false.`
- `chargesMM= [logical] Default .false.`
- `printMM= [logical] Default .false.`

9.8 Miscellaneous options

- `MOFractionOccupation= [float] Default 1.0_8`
- `ionizeMO= [integer] Default 0`
- `ionizeSpecies= [character] Default "NONE"`
- `exciteSpecies= [character] Default "NONE"`

9.9 Integrals transformation

- `integralsTransformationMethod= [character] Default "C"`
- `ITBuffersize= [integer] Default 1024`

9.10 Libraries

- uffParametersDataBase= *[character] Default "/dataBases/uffParameters.lib"*
- atomicElementsDataBase= *[character] Default "/dataBases/atomicElements.lib"*
- basisSetDataBase= *[character] Default "/basis/"*
- potentialsDataBase= *[character] Default "/potentials/"*
- elementalParticlesDataBase= *[character] Default "/dataBases/elementalParticles.lib"*

OUTPUTS

Besides the standard output, openLowdin can generate other type of outputs to view the results of an APMO calculation. Lowdin input has an “OUTPUTS” block to request these outputs. Currently, it can generate:

- Molden files for each species.
- AIM files. These files are generated with the molden2AIM program
- Gaussian cubes for orbitals and density
- Gnuplot 2D and 3D graphs for density and orbitals
- Gaussian fchk files

If CI or NOCI calculations with “CIStatesToPrint” greater than zero were selected, the molden, cubes and gnuplot files will use the CI or NOCI natural orbitals and/or density. Adding “state=N” to a line inside the “OUTPUTS” block allow us to select the desired excited state.

10.1 Molden and AIM files

To generate molden or AIM files simply add in the OUTPUTS block:

- moldenFile
- wfnFile
- wfxFile
- NBO47File

openLowdin will generate an .molden, .wfn, .wfx, or .47 file for each quantum species in the input.

For molden, there are three format types that can be selected with the CONTROL option

Table 1: moldenFileFormat =

QUANTUM	Define the coordinates, GTO and MO for each quantum species individually.
STANDARD	Same that QUANTUM but including the coordinates of classical particles. (Default)
MIXED	Same that STANDARD but including 1s GTO for each classical particles with zero contribution in the MO.

All three formats work with the MOLDEN software. Other visualization codes may require the MIXED or QUANTUM formats.

See *Molden examples* for full input examples to generate molden files

10.2 Gaussian Cubes

openLowdin generates Gaussian density or orbital cubes of a chosen species. These cubes can be read by many visualization programs, such as VMD. To generate cubes, add in the OUTPUTS block the lines

- orbitalCube
- densityCube

openLowdin will generate a .cube file for each cube requested.

Add in each line “species=symbol”, where symbol is the quantum species to be plotted. To generate a cube for each species, use “species=ALL”. For orbital plots, select an orbital with “orbital=N”. The default is the HOMO of each species.

The position of the center of the cube is declared with “center=X Y Z” or with “point1=X Y Z”. With “cubeSize=N” we declare the length of one side of the cube. The number of points is controlled either by defining the number of points per side with “pointsPerDim=N”, or the separation between points in one dimension, with “scanStep=N”

See [Cubes examples](#) for full input examples to generate orbital and density cube files

10.3 Gnuplot 2D and 3D graphs

openLowdin generates plots of the density or orbitals of a chosen species using Gnuplot. To do this, add in the OUTPUTS block the lines

- densityPlot
- orbitalPlot

As with the orbital cubes “species=symbol” selects the desired quantum species to be plotted. species=ALL is supported. For orbital plots, choose an orbital with “orbital=N”, the default will be the HOMO of each species.

To create 2D plots, add dimensions=2, select an axis and provide TWO endpoints.

- axis="A" limitA=p1 p2, replace A with x, y, z

When using the axis directive, “offsetX=”, “offsetY=”, “offsetZ=” may be used to shift the plot.

A more general definition may be provided with

- point1=X1 Y1 Z1 point2=X2 Y2 Z2, where X,Y,Z are coordinates of each endpoint

For example, to plot the electronic density from -2.0 to 2.0 along the Z axis add one of the following lines

- densityPlot species="e-" dimensions=2 axis="z" limitZ=-2.0 2.0
- densityPlot species="e-" dimensions=2 point1= 0.0 0.0 -2.0 point2=0.0 0.0 2.0

To create 3D plots, set dimensions=3 and provide a plane with two endpoints for each axis

- plane="AB" limitA=p1 p2 limitB=p3 p4, replace A and B with x, y, z

an offset along the unused axis may be selected to shift the plot.

A more general definition is also possible, where the user provides THREE points corresponding to the corners of a rectangle.

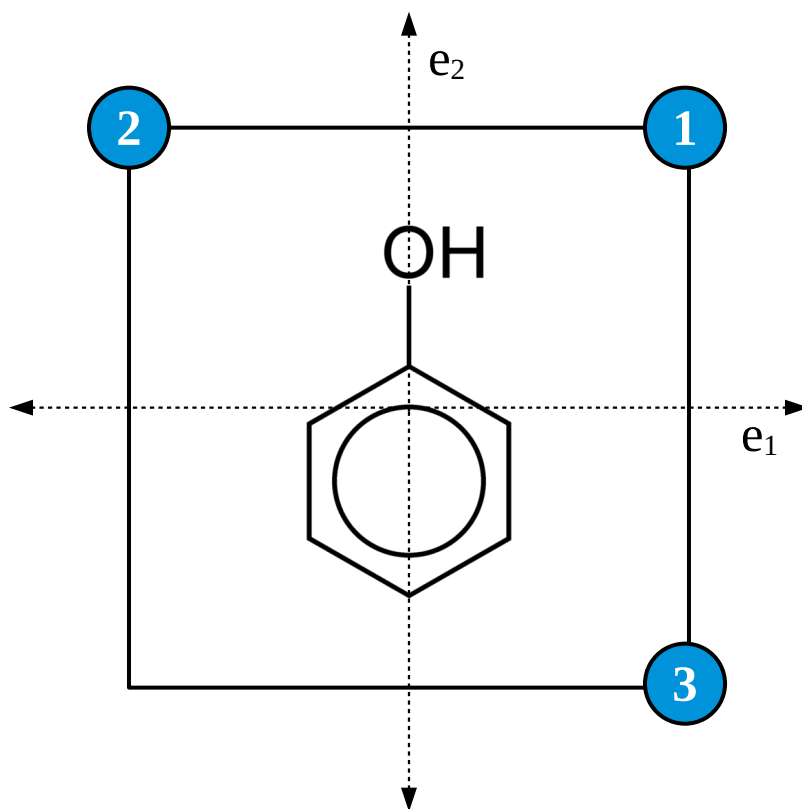
- point1=X1 Y1 Z1 point2=X2 Y2 Z2 point3=X3 Y3 Z3, where X,Y,Z are coordinates of each corner

For example, to plot electron density in the YZ plane, from -2.5 to 2.5 in both axis, use either of these lines

- densityPlot species="e-" dimensions=3 plane=yz limitY=-2.5 2.5 limitZ=-2.5 2.5

- densityPlot species="e-" dimensions=3 point1=0.0 2.5 2.5 point2=0.0 -2.5 2.5 point3=0.0 2.5 -2.5

In the general approach, the order of the corners is important. point1 must be the central coorner. See the following figure for an example



In both 2D and 3D plots, the number of points is controled either by defining the number of points per side with “pointsPerDim=N”, or the separation between points in one dimension, with “scanStep=N”

openLowdin will generate three files for Gnuplot: A .dens or a .orb with the raw data of the plot; a .gnp with the gnuplot script to generate the plot; and a .eps with the plot itself.

10.4 Fchk files

Gaussian fchk files may be used to pass the results to other programs. Currenty, openLowdin uses them to exchange information with Erkale for orbital localization. Add the line in the OUTPUTS

- fchkFile

to generate a file for each quantum species in the input. See [Molden examples](#) for an input example that employs orbital localization.

INTEGRALS

openLOWDIN computes analitically one-body and two-body integrals based on atomic orbitals expressed as a sum of contracted GTFs under a cartesian coordinate representation (see [Basis](#))

$$\phi_i^\alpha(\mathbf{r}_i) = \sum_{\mu}^{N_{bas}^\alpha} C_{\mu}^{\alpha} \varphi_{\mu}^{\alpha}(\mathbf{r}_i; \mathbf{R}_{\mu}) \quad (11.1)$$

In matrix form, the following integrals are implemented in openLOWDIN

11.1 One-body integrals

All computed under an Obara-Saika scheme for any angular momentum in the basis set

Overlap

$$S_{\mu\nu}^{\alpha} = \int dr_i \varphi_{\mu}^{\alpha}(i) \varphi_{\nu}^{\alpha}(i) \quad (11.2)$$

Kinetic

$$T_{\mu\nu}^{\alpha} = \int dr_i \varphi_{\mu}^{\alpha}(i) \nabla_i^2 \varphi_{\nu}^{\alpha}(i) \quad (11.3)$$

Point charge potential

$$\mathbb{P}\mathbb{C}_{\mu\nu}^{\alpha} = \sum_I \int dr_i \varphi_{\mu}^{\alpha}(i) \frac{1}{r_i - R_I} \varphi_{\nu}^{\alpha}(i) \quad (11.4)$$

Moment integrals

$$\mathbb{U}\mathbb{A}_{\mu\nu}^{\alpha} = \int dr_i \varphi_{\mu}^{\alpha}(i) r_{i,\mathbb{A}} \varphi_{\nu}^{\alpha}(i) \quad \mathbb{A} = x, y, z \quad (11.5)$$

$$\mathbb{U}\mathbb{A}\mathbb{B}_{\mu\nu}^{\alpha} = \int dr_i \varphi_{\mu}^{\alpha}(i) r_{i,\mathbb{A}} r_{i,\mathbb{B}} \varphi_{\nu}^{\alpha}(i) \quad \mathbb{A}, \mathbb{B} = x, y, z \quad (11.6)$$

Harmonic integrals

$$\mathbb{H}\mathbb{A}_{\mu\nu}^{\alpha} = \int dr_i \varphi_{\mu}^{\alpha}(i) r_i^2 \varphi_{\nu}^{\alpha}(i) \quad (11.7)$$

Three-center integrals

$$\mathbb{K}\mathbb{C}_{\mu\nu}^{\alpha} = \sum_{\sigma} \int dr_i \varphi_{\mu}^{\alpha}(i) \varphi_{\sigma}(i) \varphi_{\nu}^{\alpha}(i) \quad (11.8)$$

11.2 Two-body integrals

These integrals are computed with LIBINT library <https://github.com/evaleev/libint>

Four-center intraspecies coulomb potential

$$\nexists C_{\mu\nu\sigma\lambda}^{\alpha} = \langle \mu^{\alpha} \nu^{\alpha} | \sigma^{\alpha} \lambda^{\alpha} \rangle \quad (11.9)$$

$$= \int \int dr_i dr_j \varphi_{\mu}^{\alpha}(i) \varphi_{\nu}^{\alpha}(j) \frac{1}{r_i - r_j} \varphi_{\sigma}^{\alpha}(i) \varphi_{\lambda}^{\alpha}(j) \quad (11.10)$$

Four-center interspecies coulomb potential

$$\nexists C_{\mu\nu\sigma\lambda}^{\alpha\beta} = \langle \mu^{\alpha} \nu^{\beta} | \sigma^{\alpha} \lambda^{\beta} \rangle \quad (11.11)$$

$$= \int \int dr_i dr_j \varphi_{\mu}^{\alpha}(i) \varphi_{\nu}^{\beta}(j) \frac{1}{r_i - r_j} \varphi_{\sigma}^{\alpha}(i) \varphi_{\lambda}^{\beta}(j) \quad (11.12)$$

Five-center, intra- and interspecies

$$\nexists C_{\mu\nu\sigma\lambda}^{\alpha,\beta} = \langle \mu^{\alpha} \nu^{\beta} | V_2^{\alpha\beta}(\mathbf{r}_i^{\alpha}, \mathbf{r}_j^{\beta}) | \sigma^{\alpha} \lambda^{\beta} \rangle \quad (11.13)$$

$$= \sum_{\tau} C_{\tau}^{\alpha\beta} \int \int dr_i dr_j \varphi_{\mu}^{\alpha}(i) \varphi_{\nu}^{\beta}(j) \exp[-a_{\tau}^{\alpha\beta}(r_i - r_j)^2] \varphi_{\sigma}^{\alpha}(i) \varphi_{\lambda}^{\beta}(j) \quad (11.14)$$

11.2.1 Input options

- `tv= [float]` deprecated *Default 1.0E-6*
- `integralThreshold= [float]` threshold to store integrals in disk above the given value. *Default 1.0E-10*
- `integralStackSize= [integer]` write and load integrals temporary files by stacks of this values. *Default 30000*
- `integralStorage= [character]` select storage scheme for two-particles integrals

"DISK"	Storage all non-zero integrals in disk at \$SCRATCH folder, after four-index permutational symmetries. <i>Default</i>
"MEMORY"	Allocate a four dimensional array in RAM memory
"DIRECT"	Compute integrals on-the-fly (only for the SCF step)

- `integralScheme= [character]` select two-particles library. *Default "LIBINT"*
- `schwarzInequality= [logical]` perfoms Schwarz inequality to skip blocks of small integrals. Deprecated, now it's used by default within libint interface. *Default .false.*

HARTREE-FOCK, HF

The APMO/HF level wave function for a multispecies molecular system in the ground state, Ψ_0 , is constructed as a product of a single-configurational wave function, Φ_α , :

$$\Psi_0 = \prod_{\alpha}^{N_{sp}} \Phi_{\alpha}$$

In the most common case, Φ_α is built as a single slater determinant of single-particle spin molecular orbitals χ_i as

$$\Psi_{SD}(\bar{\mathbf{x}}) = \frac{1}{\sqrt{N_\alpha!}} \begin{vmatrix} \chi_1(\mathbf{x}_1) & \chi_1(\mathbf{x}_2) & \dots & \chi_1(\mathbf{x}_{N_\alpha}) \\ \chi_2(\mathbf{x}_1) & \chi_2(\mathbf{x}_2) & \dots & \chi_2(\mathbf{x}_{N_\alpha}) \\ \dots & \dots & \dots & \dots \\ \chi_{N_\alpha}(\mathbf{x}_1) & \chi_{N_\alpha}(\mathbf{x}_2) & \dots & \chi_{N_\alpha}(\mathbf{x}_{N_\alpha}) \end{vmatrix},$$

here α symbol is used to represent quantum species, $\mathbf{x}$ corresponds to their spin-coordinates vector, and $\bar{\mathbf{x}}$ is the full set of $4N$ coordinates. The spatial part of these spin orbitals, ϕ , is written as a linear combination of atomic orbitals φ with a basis set of size B (notice that the number of atomic centers can be greater than the number of classical nuclei N_c) as

$$\phi_i^\alpha(\mathbf{r}_i) = \sum_{\mu}^{N_{bas}^\alpha} C_{\mu}^\alpha \varphi_{\mu}^\alpha(\mathbf{r}_i; \mathbf{R}_{\mu})$$

Commonly these atomic orbitals are constructed with GTFs that depend on the spatial coordinate of one single particle r_i and are centered on a fixed position R for each atomic center. See [Basis](#)

The molecular orbitals are obtained by solving the coupled Fock equations, see [SCF](#),

$$f^\alpha(i) \phi_i^\alpha = \varepsilon_i^\alpha \phi_i^\alpha,$$

where ε_i are the single particle orbital energies. The effective one-particle Fock operators, $f^\alpha(i)$, for the quantum species e^- and e^+ are expanded as

$$f^\alpha(i) = h^\alpha(i) + \sum_j^{N_\alpha} (q^\alpha)^2 [J_j^\alpha - K_j^\alpha] + \sum_{\beta \neq \alpha}^{N_{sp}} \sum_j^{N_\beta} q^\alpha q^\beta J_j^\beta.$$

In the above equation $h^\alpha(i)$ is the single-particle core Hamiltonian

$$h^\alpha(i) = -\frac{1}{2m_\alpha} \nabla_i^2 + \sum_J^{N_c} \frac{q^\alpha Z_J}{R_{iJ}},$$

and J^α and K^α are the Coulomb and exchange operators defined as

$$J_j^\alpha(1) \phi_i^\alpha(1) = q^\alpha q^\alpha \left[\int d\mathbf{r}_2 \phi_j^{\alpha*}(2) \frac{1}{r_{12}} \phi_j^\alpha(2) \right] \phi_i^\alpha(1),$$

$$K_j^\alpha(1)\phi_i^\alpha(1) = \left[\int d\mathbf{r}_2 \phi_j^{\alpha*}(2) \frac{1}{r_{12}} \phi_i^\alpha(2) \right] \phi_j^\alpha(1).$$

In addition, J^β is the operator which accounts for the Coulomb potential between particles of different quantum species, thus is the term which couples the electronic and positronic Fock equations, and is defined as

$$J_j^\beta(1)\phi_i^\alpha(1) = q^\beta q^\alpha \left[\int d\mathbf{r}_2 \phi_j^{\beta*}(2) \frac{1}{r_{12}} \phi_j^\beta(2) \right] \phi_i^\alpha(1).$$

In openLOWDIN, these expressions are implemented in a matrix form [1]

$$\begin{aligned} S_{\mu\nu}^\alpha &= \int dr_1 \varphi_\mu^\alpha(1) \varphi_\nu^\alpha(1) \\ F_{\mu\nu}^\alpha &= H_{\mu\nu}^\alpha + q^\alpha q^\alpha G_{\mu\nu}^\alpha + \sum_{\beta \neq \alpha}^{N_{sp}} q^\alpha q^\beta G_{\mu\nu}^\beta \\ H_{\mu\nu}^\alpha &= \int dr_1 \varphi_\mu^\alpha(1) h^\alpha(1) \varphi_\nu^\alpha(1) \\ G_{\mu\nu}^\alpha &= \sum_{\lambda\sigma} P_{\lambda\sigma}^\alpha [(\mu^\alpha \nu^\alpha | \sigma^\alpha \lambda^\alpha) - (1/2) (\mu^\alpha \lambda^\alpha | \sigma^\alpha \nu^\alpha)] \\ G_{\mu\nu}^\beta &= \sum_{\lambda\sigma} P_{\lambda\sigma}^\beta (\mu^\alpha \nu^\alpha | \sigma^\beta \lambda^\beta), \end{aligned}$$

where $S_{\mu\nu}$, $F_{\mu\nu}$, $H_{\mu\nu}$, and $G_{\mu\nu}$ correspond to the overlap, one-core Hamiltonian, and two-particles matrices, which all run over the total number of atomic orbitals μ, ν . Here, the chemistry notation of two-particles integrals $(\varphi_\mu \varphi_\nu | \varphi_\sigma \varphi_\lambda)$ has been simplified to $(\mu\nu | \sigma\lambda)$. See [Integrals](#).

In addition, the density matrix elements are defined from the coefficient matrix $\mathbf{C}$ of the molecular orbitals expansion and the fermionic orbital occupation η

$$P_{\mu\nu}^\alpha = \eta^\alpha \sum_{\lambda}^{occ^\alpha} C_{\mu\lambda}^\alpha C_{\lambda\nu}^{\alpha*}.$$

These coefficient matrices are found by diagonalizing the Roothan-Hall equations

$$\mathbf{FC} = \epsilon \mathbf{SC}.$$

Finally, the total Hartree-Fock energy is computed from

$$E_0 = \frac{1}{2} \sum_{\alpha}^{N_{sp}} \sum_{\mu\nu}^{occ^\alpha} P_{\mu\nu}^\alpha (H_{\mu\nu}^\alpha + F_{\mu\nu}^\alpha).$$

- frozen= [character] Default "NONE"
- freezeNonElectronicOrbitals= [logical] Default .false.
- freezeElectronicOrbitals= [logical] Default .false.
- hartreeProductGuess= [logical] Default .false.
- readCoefficients= [logical] Default .true.
- readFchk= [logical] Default .false.
- writeCoefficientsInBinary= [logical] Default .true.
- readEigenvalues= [logical] Default .false.
- readEigenvaluesInBinary= [logical] Default .true.

- writeEigenvaluesInBinary= *[logical] Default .true.*
- noSCF= *[logical] Default .false.*
- finiteMassCorrection= *[logical] Default .false.*
- removeTranslationalContamination= *[logical] Default .false.*
- buildTwoParticlesMatrixForOneParticle= *[logical] Default .false.*
- buildMixedDensityMatrix= *[logical] Default .false.*
- onlyElectronicEffect= *[logical] Default .false.*
- electronicWaveFunctionAnalysis= *[logical] Default .false.*
- isOpenShell= *[logical] Default .false.*
- getGradients= *[logical] Default .false.*
- HFprintEigenvalues= *[logical] Default .true.*
- HFprintEigenvectors= *[character] Default "OCCUPIED"*
- overlapEigenThreshold= *[float] Default 1.0E-8_8*
- electricField(:)= *[float] Default 0.0_8*
- multipoleOrder= *[integer] Default 0*

SCF

openLOWDIN has different multicomponent SCF schemes to minimise the energy of a molecular system composed of multiple quantum species. The most notable is a simultaneous approach as:

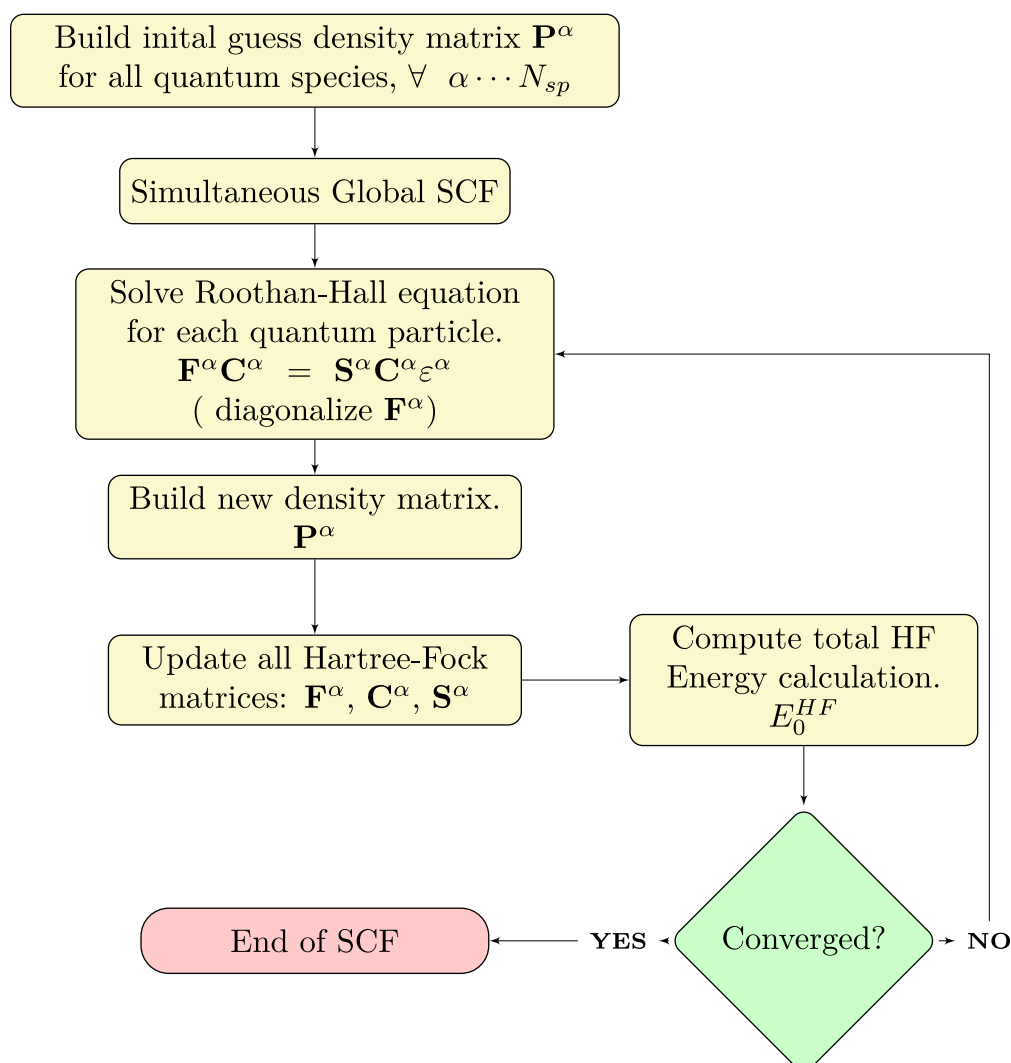


Fig. 1: Schematic of simultaneous global SCF convergence algorithm, adapted from [2]

- nonElectronicEnergyTolerance= [float] Default 1.0E-8
- electronicEnergyTolerance= [float] Default 1.0E-8

- nonelectronicDensityMatrixTolerance= *[float]* Default 1.0E-6
- electronicDensityMatrixTolerance= *[float]* Default 1.0E-6
- totalEnergyTolerance= *[float]* Default 1.0E-8
- totalDensityMatrixTolerance= *[float]* Default 1.0E-6
- densityFactorThreshold= *[float]* Default 1.0E-8
- diisSwitchThreshold= *[float]* Default 0.5
- diisSwitchThreshold_bkp= *[float]* Default 0.5
- electronicLevelShifting= *[integer]* Default 0.0
- nonelectronicLevelShifting= *[integer]* Default 0.0
- exchangeOrbitalThreshold= *[float]* Default 0.8
- waveFunctionScale= *[integer]* Default 1000.0
- scfNonelectronicMaxIterations= *[integer]* Default 50
- scfElectronicMaxIterations= *[integer]* Default 50
- scfGlobalMaxIterations= *[integer]* Default 200
- listSize= *[integer]* Default -20
- convergenceMethod= *[character]* Default 1!!(0)NONE, (1)DAMPING, (2)DIIS, (3)LEVELSHIFTING(4)DAMPING/DIIS
- diisDimensionality= *[integer]* Default 10
- iterationScheme= *[character]* Default 3!!(0)NONELECTRONICFULLY/e-(1)ELECTRONICFULLY(2)CONVERGEDINDIVIDUAL
- scfElectronicTypeGuess= *[character]* Default "HCORE"
- scfNonelectronicTypeGuess= *[character]* Default "HCORE"
- scfConvergenceCriterium= *[character]* Default "ENERGY"!ENERGY,DENSITY,BOTH
- diisErrorInDamping= *[logical]* Default .false.
- activateLevelShifting= *[logical]* Default .false.
- exchangeOrbitalsInSCF= *[logical]* Default .false.
- forceClosedShell= *[logical]* Default .false.
- debugScfs= *[logical]* Default .false.
- scfGhostSpecies= *[character]* Default "NONE"

POTENTIALS

Given all the integrals implemented in openLOWDIN (see [Integrals](#)), the code is capable of building the following potentials to the Core and Fock matrices:

Fixed point charges repulsion

$$V_{PP} = \sum_{I,J} \frac{Z_I Z_J}{R_{IJ}} \quad (14.1)$$

Quantum/fixed charges interaction

$$\hat{V}_{\text{PC}} = \sum_j \frac{q_i^\alpha Z_J}{r_i - R_J}, \quad \mathbf{V}_{\text{PC}} = q^\alpha \sum_j Z_J \mathbb{P}\mathbb{C}^\alpha \quad (14.2)$$

External electric field. It requires adding in the CONTROL section a non-zero electric field strength $\vec{F}$ for each cartesian component XYZ as a real vector `ELECTRIC_FIELD = [real real real]`

$$\sum_{\mathbb{A}}^{X,Y,Z} \hat{V}_{\mathbb{U}\mathbb{A}} = \sum_{\mathbb{A}}^{X,Y,Z} \sum_i^{N_\alpha} q_i^\alpha F_{\mathbb{A}} \cdot r_{i,\mathbb{A}}, \quad \sum_{\mathbb{A}}^{X,Y,Z} \mathbf{V}_{\mathbb{U}\mathbb{A}} = \sum_{\mathbb{A}}^{X,Y,Z} q^\alpha F_{\mathbb{A}} \mathbb{U}\mathbb{A}^\alpha \quad (14.3)$$

Harmonic potential, centered at the origin of coordinates. It requires to declare non-zero frequency ω in the GEOMETRY block with the flag `omega = [real]`

$$\hat{V}_{\mathbb{H}\mathbb{A}} = \sum_i^{N_\alpha} \frac{1}{2} m^\alpha (\omega^\alpha)^2 r_i^2, \quad \mathbf{V}_{\mathbb{H}\mathbb{A}} = \frac{1}{2} m^\alpha (\omega^\alpha)^2 \mathbb{H}\mathbb{A}^\alpha \quad (14.4)$$

Quantum Drude oscillator potential. It requires to declare non-zero frequency ω in the GEOMETRY block with the flag `omega = [real]`, as well as declaring a fixed point charge as the QDO charged center associated to the quantum species symbol with the flag `qdoCenterOf= [character]`, adding this flag will replace the quantum/fixed Coulomb potential with this point charge. For example `H dirac 0.00 0.00 0.00 qdoCenterOf=EA-` (See [Quantum Drude Oscillators, QDOs](#))

$$\hat{V}_{QDO} = \sum_i^{N_\alpha} \frac{1}{2} m^\alpha (\omega^\alpha)^2 (r_i - R_{QDO}^\alpha)^2, \quad \mathbf{V}_{QDO} = \frac{1}{2} m^\alpha (\omega^\alpha)^2 \mathbb{H}\mathbb{A}^\alpha \quad (14.5)$$

External potential. See [External potential basis](#)

$$\hat{V}_1^\alpha = \sum_i^{N_\alpha} \sum_\tau^{N_{bas}^\alpha} C_\tau^\alpha (x_i - X_\tau)^{l_\tau^\alpha} (y_i - Y_\tau)^{m_\tau^\alpha} (z_i - Z_\tau)^{n_\tau^\alpha} \times \exp[-a_\tau^\alpha (\mathbf{r}_i - \mathbf{R}_\tau)^2], \quad \mathbf{V}_1^\alpha = \mathbb{K}\mathbb{C}^\alpha \quad (14.6)$$

Intraspecies repulsion

$$\hat{V}_2^{\alpha\alpha} = \sum_{i \neq j}^{N_\alpha} \frac{q_i^\alpha q_j^\alpha}{r_i - r_j}, \quad \mathbf{V}_2^{\alpha\alpha} = q^\alpha q^\alpha G^\alpha \quad (14.7)$$

Interspecies repulsion

$$\sum_{\beta \neq \alpha}^{N_{sp}} \hat{V}_2^{\alpha\beta} = \sum_{\beta \neq \alpha}^{N_{sp}} \sum_i^{N_\alpha} \sum_j^{N_\beta} \frac{q_i^\alpha q_j^\alpha}{r_i - r_j}, \quad \sum_{\beta \neq \alpha}^{N_{sp}} \mathbf{V}_2^{\alpha\beta} = \sum_{\beta \neq \alpha}^{N_{sp}} q^\alpha q^\beta G^\beta \quad (14.8)$$

Internal potential. See [Internal potential basis](#). Notice this potential will replace the Coulomb intraspecies or interspecies potential.

$$V_2^{\alpha\beta} = \sum_i^{N_\alpha} \sum_j^{N_\beta} \sum_\tau^{N_{bas}^{\alpha\beta}} C_\tau^{\alpha\beta} \exp[-a_\tau^{\alpha\beta} (\mathbf{r}_i^\alpha - \mathbf{r}_j^\beta)^2], \quad \mathbf{V}_2^{\alpha\beta} = \nexists \mathbb{C}^\alpha \quad (14.9)$$

CHAPTER
FIFTEEN

PROPERTIES

DENSITY FUNCTIONAL THEORY, DFT

- `gridStorage= [character] Default "DISK"`
- `electronCorrelationFunctional= [character] Default "NONE"`
- `electronExchangeFunctional= [character] Default "NONE"`
- `electronExchangeCorrelationFunctional= [character] Default "NONE"`
- `nuclearElectronCorrelationFunctional= [character] Default "NONE"`
- `positronElectronCorrelationFunctional= [character] Default "NONE"`
- `betaFunction= [character] Default "NONE"`
- `gridRadialPoints= [integer] Default 35`
- `gridAngularPoints= [integer] Default 110`
- `gridNumberOfShells= [integer] Default 5`
- `finalGridRadialPoints= [integer] Default 50`
- `finalGridAngularPoints= [integer] Default 302`
- `finalGridNumberOfShells= [integer] Default 5`
- `polarizationOrder= [integer] Default 1`
- `numberOfBlocksInAuxiliaryFunctions= [integer] Default 3`
- `fukuiFunctions= [logical] Default .false.`
- `auxiliaryDensity= [logical] Default .false.`
- `storeThreeCenterElectronIntegrals= [logical] Default .true.`
- `callLibxc= [logical] Default .true.`
- `nuclearElectronDensityThreshold= [float] Default 1E-10`
- `electronDensityThreshold= [float] Default 1E-10`
- `gridWeightThreshold= [float] Default 1E-10`
- `betaParameterA= [integer] Default 0.0`
- `betaParameterB= [integer] Default 0.0`
- `betaParameterC= [integer] Default 0.0`

16.1 Subsystem embedding Options

- `subsystemEmbedding= [logical] Default .false.`
- `localizeOrbitals= [logical] Default .false.`
- `subsystemLevelShifting= [integer] Default 1.0E6`
- `subsystemOrbitalThreshold= [float] Default 0.1`
- `subsystemBasisThreshold= [float] Default 0.0001`
- `erkaleLocalizationMethod= [character] Default "MU"`

MANY-BODY PERTURBATION THEORY, MBPT

OpenLowdin can perform second order many-body perturbation theory (MP2) corrections to the energy on systems including any kind of quantum particle. These corrections are probably the simplest way to recover the effects of correlation between particles of different species.

$$\begin{aligned} E_{\text{MP2}} &= E_{\text{HF}} + \sum_{\alpha}^{N_{\text{typ}}} E_{\alpha\alpha}^{(2)} + \sum_{\alpha\beta}^{N_{\text{typ}}} E_{\alpha\beta}^{(2)}, \\ E_{\alpha\alpha}^{(2)} &= \frac{q_{\alpha}^2}{4} \sum_{ij}^{oc_{\alpha}} \sum_{ab}^{vir_{\alpha}} \frac{|\langle i_{\alpha} j_{\alpha} || a_{\alpha} b_{\alpha} \rangle|^2}{\varepsilon_{\alpha i} + \varepsilon_{\alpha j} - \varepsilon_{\alpha a} - \varepsilon_{\alpha b}}, \\ E_{\alpha\beta}^{(2)} &= q_{\alpha} q_{\beta} \sum_i^{oc_{\alpha}} \sum_j^{oc_{\beta}} \sum_a^{vir_{\alpha}} \sum_b^{vir_{\beta}} \frac{|\langle i_{\alpha} j_{\beta} || a_{\alpha} b_{\beta} \rangle|^2}{\varepsilon_{\alpha i} + \varepsilon_{\beta j} - \varepsilon_{\alpha a} - \varepsilon_{\beta b}}. \end{aligned} \tag{17.1}$$

For more information on APMO-MP2 calculations see S. A. Gonz'alez, A. Reyes, "Nuclear Quantum Effects on the He2H+ Complex With the Nuclear Molecular Orbital Approach", Int. J. Quant. Chem. 110 689 (2010) <https://doi.org/10.1002/qua.22118>

In addition, OpenLowdin can compute second order Epstein-Nesbet (EN2) corrections, which correspond to renormalized MP2 equations.

See *MP2 with quantum nucleus* for an example of a MP2 calculation in openLowdin

- `mpCorrection= [integer] Default 1`
- `mpFrozenCoreBoundary= [integer] Default 0`
- `mpOnlyElectronicCorrection= [logical] Default .false.`
- `epsteinNesbetCorrection= [integer] Default 1`

PROPAGATOR THEORY, PT

OpenLOWDIN can calculate ionization potentials for any species employing the propagator formalism, where ionization energy for an specific orbital is calculated as the Koopmans value plus self-energy corrections. The current implementation includes second order corrections (APMO/P2), second order plus transition operator corrections (APMO/TOEP2), third order corrections (APMO/P3) and renormalized third order corrections (APMO/REN-P3) as in the multicomponent extension of the outer valence Green function method (APMO/OVGF)

At second order, there are intraspecies and interspecies contributions to the self energy corrections to the eigenvalue of reference orbital p:

$$\begin{aligned}\omega_{\alpha p} &= \epsilon_{\alpha p} + \Sigma_{\alpha pp}(\omega_{\alpha p}) \\ \Sigma_{\alpha pp}^{(2)}(\omega_{\alpha p}) &= \sum_i^{oc_\alpha} \sum_{ab}^{vir_\alpha} \frac{|\langle p_\alpha i_\alpha | a_\alpha b_\alpha \rangle|^2}{\omega_{\alpha p} + \epsilon_{\alpha i} - \epsilon_{\alpha a} - \epsilon_{\alpha b}} + \sum_{ij}^{oc_\alpha} \sum_a^{vir_\alpha} \frac{|\langle p_\alpha a_\alpha | i_\alpha j_\alpha \rangle|^2}{\omega_{\alpha p} + \epsilon_{\alpha a} - \epsilon_{\alpha i} - \epsilon_{\alpha j}} \\ &\quad + \sum_a^{vir_\alpha} \sum_i^{oc_\beta} \sum_b^{vir_\beta} \frac{|\langle p_\alpha i_\beta | a_\alpha b_\beta \rangle|^2}{\omega_{\alpha p} + \epsilon_{\beta i} - \epsilon_{\alpha a} - \epsilon_{\beta b}} + \sum_i^{oc_\alpha} \sum_j^{oc_\beta} \sum_a^{vir_\beta} \frac{|\langle p_\alpha a_\beta | i_\alpha j_\beta \rangle|^2}{\omega_{\alpha p} + \epsilon_{\beta a} - \epsilon_{\alpha i} - \epsilon_{\beta j}}\end{aligned}\quad (18.1)$$

More details on the multicomponent propagator methods can be found in (romero.JCP.137.074105.2012, romero.JCP.141.114103.2014)

To perform a propagator calculations using LOWDIN, the order of PT (2 or 3) must be specified in the “TASKS” block using the keyword “propagatorTheoryCorrection”. Currently, the third order corrections is only available from a UHF reference.

A default calculation obtains ionization energies for the HOMO and LUMO orbitals of all the species present in the input. Using the “IonizeMO” and “ionizeSpecies” in the “CONTROL” block, allows the user to select specific orbitals for the propagator calculation. In that case, set up the flag “ptJustOneOrbital=.T.” to save computational time.

Transition operator corrections latter take advantage of fractional occupation to include additional relaxation in calculated ionization energies. To perform calculations with this method, select the UHF reference, add to the control block the “ptTransitionOperator=.T.” and select a fractional occupation using “MOfractionOccupation”. The recommended value is 0.5.

For third order calculations, you can select in the “CONTROL” block the type of correction to be used by adding “ptP3Method=” with “P3”, “EP3”, “OVGF-A”, “OVGF-B”, “OVGF-C” and “REN-P3” as options. By default, the six correction types are computed.

See *PT2 with quantum nucleus*, *TOEP2 with quantum nucleus* and *PT3 for a Ps-atom complex* for examples of propagator calculations in openLowdin

- ptOnlyOneSpecieCorrection= [logical] Default .false.
- selfEnergyScan= [logical] Default .false.
- ptTransitionOperator= [logical] Default .false.
- ptJustOneOrbital= [logical] Default .false.

- selfEnergySpacing= *[float]* Default 0.5_8
- selfEnergyRange= *[float]* Default 5.0_8
- ptOrder= *[integer]* Default 1
- ptMaxIterations= *[integer]* Default 50
- ptIterationMethod2Limit= *[integer]* Default 1
- ptIterationScheme= *[integer]* Default 1
- ptMaxNumberOfPolesSearched= *[integer]* Default 10
- ptFactorSS= *[integer]* Default 0
- ptFactorOS= *[integer]* Default 0
- ptP3Method= *[character]* Default "NONE"
- ptP3Method(1)= *[character]* Default "ALL"

CONFIGURATION INTERACTION, CI

19.1 CI Wavefunction

The APMO/CI wave function is written as a linear combination of CI configurations

$$|\Psi_{\text{CI}}\rangle = \sum_I^{N_{\text{conf}}} c_I |\Psi_I\rangle \quad (19.1)$$

as products of Slater determinants between all quantum species

$$|\Psi_I\rangle = \prod_{\alpha}^{N_{sp}} |D_I\rangle_{\alpha} = |D_I\rangle_{\alpha} |D_I\rangle_{\beta} |D_I\rangle_{\gamma} \dots \quad (19.2)$$

where each is built as excitations within each type of quantum species, written in second quantization as:

$$|D_I\rangle_{\alpha} = a_i^{\dagger} a_j^{\dagger} \dots a_{N_{\alpha}}^{\dagger} | \rangle_{\alpha} \quad (19.3)$$

For example,

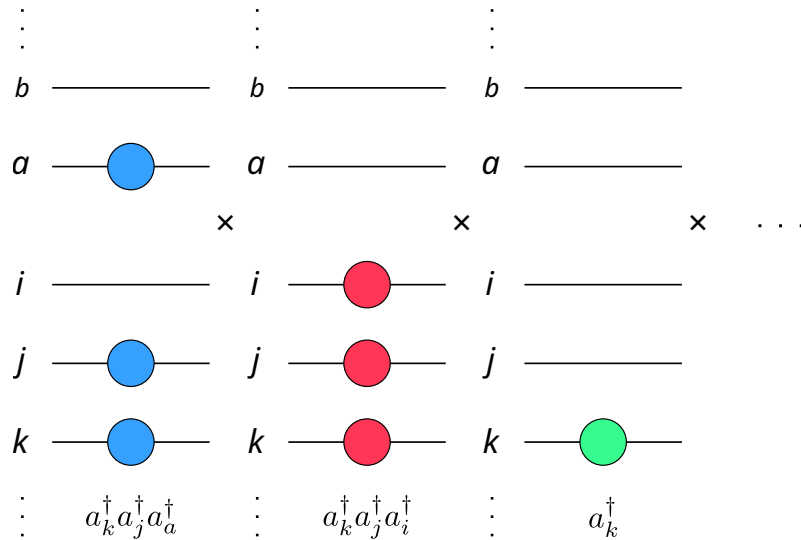


Fig. 1: Schematic of multicomponent CI configuration

Normally in CI, those configurations are group together according to the excitation level

$$|\Psi_{\text{CI}}\rangle = c_0 |\Psi_0\rangle + \sum_{\alpha} \sum_{ia \in \alpha} c_i^a |\Psi_i^a\rangle + \sum_{\alpha, \beta} \sum_{ia \in \alpha, jb \in \beta} c_{ij}^{ab} |\Psi_{ij}^{ab}\rangle + \sum_{\alpha, \beta} \sum_{ia \in \alpha, jb \in \alpha, kc \in \beta} c_{ijk}^{abc} |\Psi_{ijk}^{abc}\rangle + \dots \quad (19.4)$$

leading to all singles excitations for all species, doubles within the same species (intraspecies), doubles as products of two single excitations from two different species (interspecies) for all possible pairs, as well as triples, quadrupoles, ... and so on.

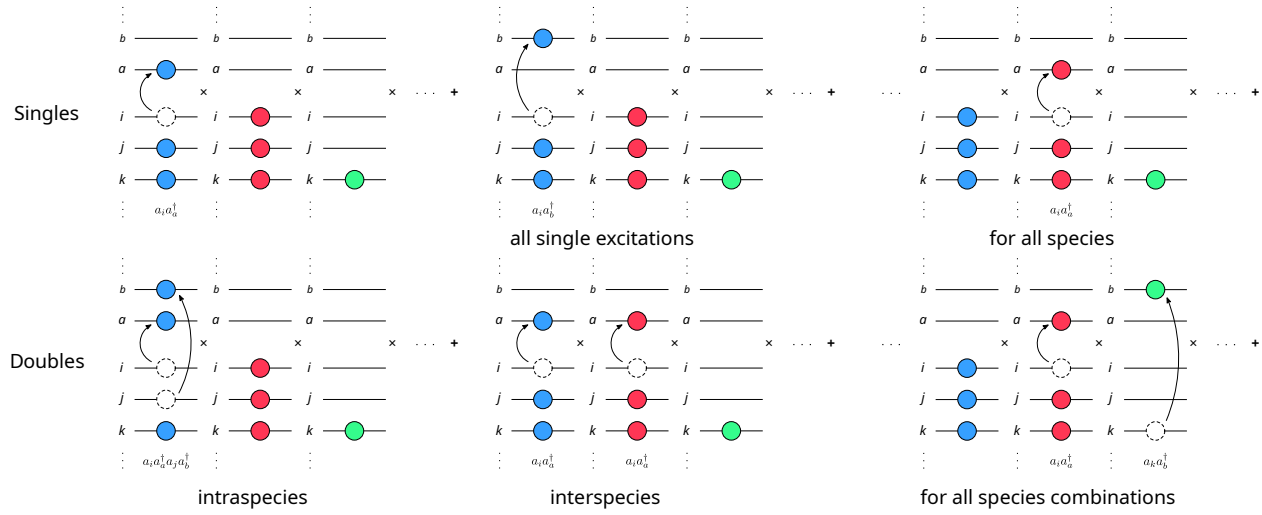


Fig. 2: Schematic of multicomponent CI wave function

Given that wavefunction, the CI Hamiltonian matrix are computed following Slater's rules taking into account that the multispecies potential operator are still a two-body operator at most.

Identical determinants:

$$\langle \Phi_I | \hat{H} | \Phi_I \rangle = \sum_{\alpha} N_s \sum_i^{N^{\alpha}} [i | \hat{h} | i] + \sum_{\alpha} \sum_i^{N^{\alpha}} \sum_{j>i}^{N^{\alpha}} ([ii|jj] - [ij|ji]) + \sum_{\alpha} \sum_{\beta>\alpha}^{N^{\alpha} N^{\beta}} \sum_{\substack{i \in \alpha \\ j \in \beta}} [ii|jj] \quad (19.5)$$

One spin-orbital different

$$|\Phi_I\rangle = |\cdots i_{\alpha} \cdots\rangle \prod_{\beta \neq \alpha}^{N_s} |\beta(I)\rangle \quad |\Phi_J\rangle = |\cdots j_{\alpha} \cdots\rangle \prod_{\beta \neq \alpha}^{N_s} |\beta(I)\rangle \quad (19.6)$$

$$\langle \Phi_I | \hat{H} | \Phi_J \rangle = [i | \hat{h} | j] + \sum_k^{N^{\alpha}} ([ij|kk] - [ik|kj]) + \sum_{\beta \neq \alpha} \sum_{k \in \beta}^{N^{\beta}} [ij|kk], \quad i, j \in \alpha \quad (19.7)$$

Two spin-orbital different, same species

$$|\Phi_I\rangle = |\cdots i_{\alpha} j_{\alpha} \cdots\rangle \prod_{\beta \neq \alpha}^{N_s} |\beta(I)\rangle \quad |\Phi_J\rangle = |\cdots k_{\alpha} l_{\alpha} \cdots\rangle \prod_{\beta \neq \alpha}^{N_s} |\beta(I)\rangle \quad (19.8)$$

$$\langle \Phi_I | \hat{H} | \Phi_J \rangle = [ik|jl] - [il|jk], \quad i, j, k, l \in \alpha \quad (19.9)$$

One spin-orbital different for two different quantum species

$$|\Phi_I\rangle = |\cdots i_{\alpha} \cdots\rangle |\cdots k_{\beta} \cdots\rangle \prod_{\substack{\gamma \neq \alpha \\ \gamma \neq \beta}}^{N_s} |\gamma(I)\rangle \quad (19.10)$$

$$|\Phi_J\rangle = |\cdots j_{\alpha} \cdots\rangle |\cdots l_{\beta} \cdots\rangle \prod_{\substack{\gamma \neq \alpha \\ \gamma \neq \beta}}^{N_s} |\gamma(I)\rangle \quad (19.11)$$

$$\langle \Phi_I | \hat{H} | \Phi_J \rangle = [ij|kl], \quad i, j \in \alpha, k, l \in \beta \quad (19.12)$$

More than two spin orbital different within same species or more than two different species, differing in one spin orbital.

$$\langle \Phi_I | \hat{H} | \Phi_J \rangle = 0 \quad (19.13)$$

The combinational CI coefficients c are obtained variationally by solving the CI eigenvalue problem $\mathbf{H}\mathbf{c} = E\mathbf{c}$. Due to the size of the Hamiltonian Matrix $\mathbf{H}$, it is necessary to use iterative techniques for diagonalization of real symmetric matrices avoiding the memory storage of the full matrix. Davidson's method is the most used technique to find the lowest eigenvalues and eigenvectors. Here, the storage requirements of $\mathbf{H}$ can be greatly reduced by direct calculation of a matrix-vector product of the form $\mathbf{H}\mathbf{c} = \sigma$.

19.2 Selected CI, SCI

In openLOWDIN, the CI techniques aimed to select the most relevant configurations within the whole CI space are encompassed as the SCI family. These are usually categorized as stochastic or deterministic [3]. Currently the code has implemented an extended multicomponent version of Adaptive Sampling CI, ASCI [4].

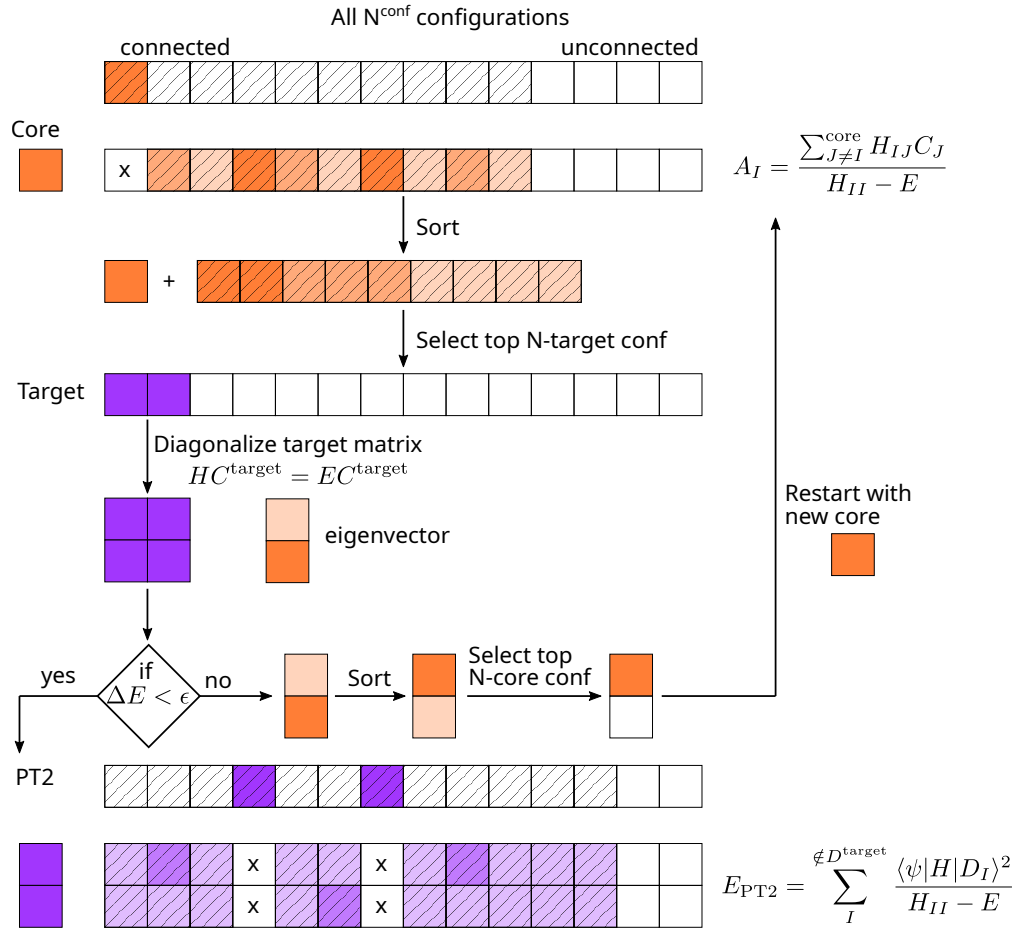


Fig. 3: Schematic of SCI algorithm

19.3 Natural orbitals

In CI, natural orbitals are special orbitals defined as the eigenfunctions of the one-particle reduced CI density. Using them significantly improves CI convergence by reducing the number of configurations needed to describe electron correlation. This allows a CI calculation to achieve good accuracy with a much smaller set of configurations, as natural orbitals focus on those with large occupation numbers. Thus, the CI reduced density matrix for the CI I -th state is defined as

$$\gamma_{\mu,\nu}^{CI(I)} = \langle \Psi^{(I)} | a_{\mu}^{\dagger} a_{\nu} | \Psi^{(I)} \rangle \quad (19.14)$$

The natural orbitals are the eigenvectors of this density matrix, and the eigenvalues of the diagonalized density matrix are the natural occupation numbers, which indicate the importance of each natural orbital in the overall wave function.

19.4 Diagonal corrections

19.5 CI Input options

- `configurationInteractionLevel=` *[character]* Select Configuration Interaction level.

"CIS"	Singles
"CISD"	Singles and doubles
"CISD-"	Singles, doubles interspecies only
"CISD+"	Singles, doubles, triples interpecies only
"CISDT"	Singles, doubles, triples
"CISDTQ"	Singles, doubles, triples, quadruples
"SCI"	Selected CI
"FCI"	Full CI
"NONE"	<i>Default</i>

- `numberOfCIStates=` *[integer]* Set the number of lowest energy eigenvectors to find for the CI matrix in increasing energy. *Default* 1 (Ground state)
- `CIdiagonalizationMethod=` *[character]* Select the diagonalization subroutine

"DSYEVX"	LAPACK subroutine for selected eigenvalues and, optionally, eigenvectors of a real symmetric matrix. It requires to precompute and hold in memory the whole matrix (NXN) <i>Default</i>
"DSYEVr"	Newest LAPACK subroutine for selected eigenvalues and, optionally, eigenvectors of a real symmetric matrix. It requires to precompute and hold in memory the whole matrix (NXN) <i>Default</i>
"JADAMILU"	Jacobi-Davidson iterative algorithm. It requires to save only m-auxiliary matrix-vector products (NXm).

- `CIdiagonalDressedShift=` *[character]* Add a diagonal shift to the CI matrix for size-extensive corrections. Values: CISD. *Default* "NONE"
- `CIstatesToPrint=` *[integer]* Number of CI states to print additional information about configuration occupations. *Default* 1
- `CIsizeOfGuessMatrix=` *[integer]* Size N of the initial CI space to initialize the first eigenvector guess, it requires to hold in memory a matrix of NXN size for its diagonalization *Default* 300

- `CIStackSize= [integer]` Stacksize of an temporary array to load and save CI eigenvectors in disk. *Default 5000*
- `CIConvergence= [float]` Tolerance threshold for the eigenvector residual in JADAMILU *Default 1E-4*
- `CImatvecTolerance= [float]` Tolerance threshold in the CI eigenvector guess for skipping the matrix-vector calculation during JADAMILU's iterations. Higher values reduce the cost of each iteration but increase the number of steps. *Default 1E-10*
- `CISaveEigenvector= [logical]` Save CI eigenvector for future computations *Default .false.*
- `CIloadEigenvector= [logical]` Load previously saved CI eigenvector as an initial guess *Default .false.*
- `CIadSpace= [integer]` Desired size of the search space in JADAMILU, MS. The memory cost of JADAMILU's diagonalization is given by $N*(3*MS+NEIG+1)+4*MS*MS$. Where N is the number of configurations, and NEIG the number of desired eigenvectors to find. *Default 5*
- `CINaturalOrbitals= [logical]` Flag to compute natural orbitals after CI diagonalization. *Default .false.*
- `CIPrintEigenvectorsFormat= [character]` Select the format to print the CI eigenvectors for each configuration above `abs(CIPrintThreshold)`

"ORBITALS"	Print all orbitals occupation (0 or 1)
"OCCUPIED"	Print the occupied orbitals index <i>Default</i>
"NONE"	No printing

- `CIPrintThreshold= [float]` Print all configurations above `abs(CIPrintThreshold)` for each eigenstate *Default 1E-1*

NON-ORTHOGONAL CONFIGURATION INTERACTION, NOCI

- `nonOrthogonalConfigurationInteraction= [logical] Default .false.`
- `translationScanGrid(:)= [integer] Default 0`
- `rotationalScanGrid= [integer] Default 0`
- `rotationAroundZMaxAngle= [integer] Default 360`
- `rotationAroundZStep= [integer] Default 0`
- `nestedRotationalGrids= [integer] Default 1`
- `translationStep= [integer] Default 0.0`
- `nestedGridsDisplacement= [integer] Default 0.0`
- `configurationEnergyThreshold= [integer] Default 1.0`
- `configurationOverlapThreshold= [float] Default 1.0E-8`
- `configurationMaxDisplacement(:)= [integer] Default 0.0`
- `configurationMinDisplacement(:)= [integer] Default 0.0`
- `configurationMaxNPDdistance= [integer] Default 1.0E8`
- `configurationMinPPDistance= [integer] Default 0.0`
- `configurationMaxPPDistance= [integer] Default 1.0E8`
- `configurationEquivalenceDistance= [float] Default 1.0E-8`
- `empiricalOverlapParameterA= [float] Default 0.0604`
- `empiricalOverlapParameterB= [float] Default 0.492`
- `empiricalOverlapParameterE0= [integer] Default 0.0`
- `empiricalOverlapParameterSc= [integer] Default 0.0`
- `configurationUseSymmetry= [logical] Default .false.`
- `readNOCIgeometries= [logical] Default .false.`
- `empiricalOverlapCorrection= [logical] Default .false.`
- `onlyFirstNOCIelements= [logical] Default .false.`
- `computeROCIformula= [logical] Default .false.`

TUTORIALS

In this section you can find multiples examples for openLOWDIN calculations

- *Positronic systems*
- *Positron covalent bond*
- *Quantum Nuclei*
- *MP2 with quantum nucleus*
- *Negative Muons*
- *Quantum Drude Oscillators, QDOs*
- *External Potentials*
- *Molden examples*
- *Cubes examples*

POSITRONIC SYSTEMS

This is an example on how to run a *Configuration Interaction, CI* calculations for PsH

```
GEOMETRY
  e-(H)  SHARON-E-6S2P      0.00      0.00      0.00 addParticles=1
  H      dirac              0.00      0.00      0.00
  e+     SHARON-E+6S2P     0.00      0.00      0.00
END GEOMETRY

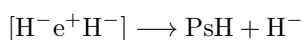
TASKS
  method = "UHF"
  configurationInteractionLevel = "FCI"
  !configurationInteractionLevel = "CISD"
END TASKS

CONTROL
readCoefficients=F
numberOfCIstates=3
CINaturalOrbitals=T
  CIStatesToPrint = 1
  !CIdiagonalizationMethod = "DSYEVX"
  CIdiagonalizationMethod = "JADAMILU"
  !CIPrintEigenVectorsFormat = "NONE"
  CIPrintEigenVectorsFormat = "OCCUPIED"
  !CIPrintEigenVectorsFormat = "ORBITALS"
  CIPrintThreshold = 5e-2
  buildTwoParticlesMatrixForOneParticle=T
END CONTROL

INPUT_CI
  species="E-ALPHA" core=0 active=0
  species="E-BETA" core=0 active=0
  species="E+" core=0 active=0
END INPUT_CI
```


POSITRON COVALENT BOND

This is an example on how to compute the binding energy of a dihydride positron-bound system, using *Configuration Interaction, CI* calculations, as was done in <https://doi.org/10.1002/anie.201800914>



This input computes the energy of the dihydride system

```
!The goal of this calculation is to compute the binding energy of a positron bound
↪complex
!Reported:
!E(e+H2^2-): -1.279372 a.u.

SYSTEM_DESCRIPTION='e+H2^2- from Charry 2018 (10.1002/anie.201800914)'

!add two electrons (one for each hydrogen anion)
!remove one positron
GEOMETRY
      e-(H)   AUG-CC-PVDZ      0.00      0.00      -1.6 addParticles=1
      e-(H)   AUG-CC-PVDZ      0.00      0.00      1.6 addParticles=1
      e+       E+-H-AUG-CC-PVDZ      0.00      0.00      -1.6
      e+       E+-H-AUG-CC-PVDZ      0.00      0.00      1.6 addParticles=-1
      H        dirac           0.00      0.00      -1.6
      H        dirac           0.00      0.00      1.6
END GEOMETRY

!method to solve the SCF - CI only works for unrestricted reference
!CI level strings chooses the desired excitations to be included. FCI is all possible
↪excitations

TASKS
      method = "UHF"
      configurationInteractionLevel ="CISDTQ"
      !configurationInteractionLevel ="CIS","CISD","CISDT","CISDTQ","FCI"
END TASKS

!Compute only the "numberOfCIstates" states. Here we select the ground and the first
↪excited state
!Compute the density matrix for "CIstatesToPrint" states, for density outputs
!Generate the natural orbitals, for visualization in molden files
!The Davidson diagonalization implemented in JADAMILU is the recommended method.
!For small systems, full matrix diagonalization with DSYEVX is possible
```

(continues on next page)

(continued from previous page)

```

!CI EigenVectors with coefficient higher than "CIPrintThreshold" are printed
!Printing format "OCCUPIED" shows the coefficients, "ORBITALS" shows the strings, "NONE"
↳ skips printing
!Strict SCF convergence improves the quality of the CI results (not required for the FCI)

CONTROL
  numberOfCIstates=2
  CIStatesToPrint=2
  CINaturalOrbitals=T
  CIdiagonalizationMethod = "JADAMILU"
  !CIdiagonalizationMethod = "DSYEVX"
  CIPrintEigenVectorsFormat = "OCCUPIED"
  !CIPrintEigenVectorsFormat = "NONE","ORBITALS"
  CIPrintThreshold = 5e-2
  totalEnergyTolerance=1E-12
END CONTROL

!INPUT_CI block help us define the frozen core and active virtuals orbitals. Here we are
↳ not restricting the excitation space
INPUT_CI
  species="E-ALPHA" core=0 active=0
  species="E-BETA" core=0 active=0
  species="E+" core=0 active=0
END INPUT_CI

!With CI, moldenFiles, 1D and 2D density slices and density cubes are good ways to
↳ visualize the density results
OUTPUTS
  moldenFile state=1
  moldenFile state=2
  densityPlot dimensions=2 point1=0.0 0.0 -6.0 point2=0.0 0.0 6.0 state=1
↳ scanStep=0.001
  densityPlot dimensions=2 point1=0.0 0.0 -6.0 point2=0.0 0.0 6.0 state=2
↳ scanStep=0.001
  densityPlot dimensions=3 point1=0.0 -3.0 -4.0 point2=0.0 -3.0 4.0 point3=0.0 3.0
↳ -4.0 state=1 scanStep=0.01
  densityPlot dimensions=3 point1=0.0 -3.0 -4.0 point2=0.0 -3.0 4.0 point3=0.0 3.0
↳ -4.0 state=2 scanStep=0.01
END OUTPUTS

```

Then, we have to subtract the energy obtained from calculations of the dissociated species

```

!The goal of this calculation is to compute the binding energy of a positron bound
↳ complex
!Reported:
!E(PsH): -0.734559

SYSTEM_DESCRIPTION='PsH from Charry 2018 (10.1002/anie.201800914)'

```

(continues on next page)

(continued from previous page)

```

GEOMETRY
    e-(H)    AUG-CC-PVDZ      0.00      0.00      0.00 addParticles=1
    e+       E+-H-AUG-CC-PVDZ  0.00      0.00      0.00
    H        dirac           0.00      0.00      0.00
END GEOMETRY

TASKS
    method = "UHF"
    configurationInteractionLevel = "FCI"
END TASKS

CONTROL
    numberOfCIstates=1
    CIStatesToPrint=1
    CINaturalOrbitals=T
    CIdiagonalizationMethod = "JADAMILU"
    CIPrintEigenvectorsFormat = "OCCUPIED"
    CIPrintThreshold = 5e-2
END CONTROL

OUTPUTS
    moldenFile state=1
    densityPlot dimensions=2 point1=0.0 0.0 -6.0 point2=0.0 0.0 6.0 state=1
END OUTPUTS

```

```

!The goal of this calculation is to compute the binding energy of a positron bound_
↪ complex
!Reported:
!E(H-): -0.524029

SYSTEM_DESCRIPTION='H- from Charry 2018 (10.1002/anie.201800914)'

GEOMETRY
    e-(H)    AUG-CC-PVDZ      0.00      0.00      0.00 addParticles=1
    H        dirac           0.00      0.00      0.00
END GEOMETRY

TASKS
    method = "UHF"
    configurationInteractionLevel = "FCI"
END TASKS

CONTROL
    numberOfCIstates=1
    CIStatesToPrint=1
    CINaturalOrbitals=T
    CIdiagonalizationMethod = "JADAMILU"

```

(continues on next page)

(continued from previous page)

```
        CIPrintEigenVectorsFormat = "OCCUPIED"
        CIPrintThreshold = 5e-2
END CONTROL

OUTPUTS
    moldenFile state=1
    densityPlot dimensions=2 point1=0.0 0.0 -6.0 point2=0.0 0.0 6.0 state=1
END OUTPUTS
```

QUANTUM NUCLEI

MP2 WITH QUANTUM NUCLEUS

A MP2 (second order *Many-Body Perturbation Theory*, *MBPT*) input on the hydrogen fluoride molecule, where the electrons and hydrogen nucleus are treated as quantum particles should be like this

```

GEOMETRY
  e-(F)  cc-pvtz  0.00 0.00 0.91
  e-(H)  cc-pvtz  0.00 0.00 0.00
  F      dirac    0.00 0.00 0.91
  H_1    dzspnb   0.00 0.00 0.00
END GEOMETRY
TASKS
  method="RHF"
  mollerPlessetCorrection=2
END TASKS
CONTROL
  mpFrozenCoreBoundary=1
END CONTROL

```

Here, APMO-MP2 calculations are performed when the option “mollerPlessetCorrection=2” is present in the “TASKS” block. MP2 calculations may use RHF or UHF as reference.

The CONTROL option mpFrozenCoreBoundary: Omits this number of occupied electronic molecular orbitals in the MP2 calculations (core electrons). Default 0.

For other species, the number of core orbitals along with the number of active virtuals, can be controlled with the “INPUT_CI” block in the input.

A MP2 output will include the summary of the MP2 results

```

POST HARTREE-FOCK CALCULATION
MANY-BODY PERTURBATION THEORY:
=====

      MOLLER-PLESSET FORMALISM
      ORDER OF CORRECTION =      2

      E(0) + E(1) =    -100.017935705706
      E(2) =      -0.283903659181
      -----
      E(MP2) =    -100.301839364887

      -----
      E(n){ Species }      E(n) / Hartree

```

(continues on next page)

(continued from previous page)

```
-----  
      E(2){ E- } =      -0.273701262404  
      E(2){ H_1 } =      0.000000000000  
  
      E(2){ E-/H_1 } =    -0.010202396777
```

Where the E(2) is the second order correction to the energy, and the E(MP2) is the Hartree-Fock energy plus E(2). This summary also includes the intraspecies and interspecies contributions to E(2).

PT2 WITH QUANTUM NUCLEUS

This is a minimal input for a second order propagator theory (*Propagator Theory, PT*) calculation to obtain the electronic ionization energies and proton binding energies for a water molecule. The PT2 corrections are computed when “propagatorTheoryCorrection=2” is added to the input:

```

GEOMETRY
e-(O)      6-31G      0.0000  0.0000  0.1173
e-(H)      6-31G      0.0000  0.7572 -0.4692
e-(H)      6-31G      0.0000 -0.7572 -0.4692
O          dirac      0.0000  0.0000  0.1173
H-a_1      Nakai-7-SPD 0.0000  0.7572 -0.4692
H-b_1      Nakai-7-SPD 0.0000 -0.7572 -0.4692
END GEOMETRY

TASKS
      method = "RHF"
      propagatorTheoryCorrection=2
END TASKS

```

The PT2 output will include a summary of the Koopmans’ (KT) and self-energy corrected (EP2) results for the highest occupied and lowest unoccupied orbital of each species,

```

POST HARTREE-FOCK CALCULATION
PROPAGATOR THEORY:
=====

PROPAGATOR FORMALISM FOR SEVERAL FERMIONS SPECIES
ORDER OF CORRECTION =      2
The following articles must be cited:
-----
      J. Chem. Phys. 137, 074105 (2012)
      J. Chem. Phys. 138, 194108 (2013)
                        CPL coming soon
-----

SPECIES:      E-
-----
      Orbital      KT (eV)      EP2 (eV)      P.S
-----
           5      -13.335244    -10.140654    0.853889
           6       5.362715      4.925340     0.952097
-----

SPECIES:      H-A_1

```

(continues on next page)

(continued from previous page)

Orbital	KT (eV)	EP2 (eV)	P.S
1	-24.603421	-17.007032	0.877287
2	-23.948321	-57.734065	0.538455
SPECIES: H-B_1			
Orbital	KT (eV)	EP2 (eV)	P.S
1	-24.603421	-17.007032	0.877287
2	-23.948321	-57.734065	0.538455

For the occupied orbitals, KT and EP2 results are estimates of the energy required to remove a particle from the corresponding orbital (ionization potential). For the unoccupied ones, KT and EP2 results are estimates of the energy gained by adding a particle to the orbital (electron affinity). Here, the Pole Strength (P.S) serves as a quantity that validates the diagonal (pseudoparticle) approximation employed. A P.S value below 0.85 usually indicates that the diagonal approximation is not reliable.

TOEP2 WITH QUANTUM NUCLEUS

Here, we compute the second order propagator theory (*Propagator Theory, PT*) corrections to a partially ionized water molecule electronic orbital (HOMO). The TOEP2 method requires a UHF reference.

First, run a regular UHF calculation to generate the molecular orbitals, let's name it "H2O.UHF.lowdin". These molecular orbitals will be used as the guess for the partially ionized SCF.

```
GEOMETRY
  e-(O)      6-31G      0.0000  0.0000  0.1173 multiplicity=1
  e-(H)      6-31G      0.0000  0.7572 -0.4692
  e-(H)      6-31G      0.0000 -0.7572 -0.4692
  O          dirac      0.0000  0.0000  0.1173
  H-a_1      Nakai-7-SPD 0.0000  0.7572 -0.4692
  H-b_1      Nakai-7-SPD 0.0000 -0.7572 -0.4692
END GEOMETRY

TASKS
  method = "UHF"
END TASKS
```

Now, we generate the TOEP2 input, let's name it "H2O.TOEP2.lowdin". In addition to the PT2 flag, "propagatorTheoryCorrection=2", we add to the input the flag "ptTransitionOperator=T" along with "IonizeMO" and "ionizeSpecies" to select the orbital and the species, and "MOfractionOccupation" to select the occupation.

```
GEOMETRY
  e-(O)      6-31G      0.0000  0.0000  0.1173 multiplicity=1
  e-(H)      6-31G      0.0000  0.7572 -0.4692
  e-(H)      6-31G      0.0000 -0.7572 -0.4692
  O          dirac      0.0000  0.0000  0.1173
  H-a_1      Nakai-7-SPD 0.0000  0.7572 -0.4692
  H-b_1      Nakai-7-SPD 0.0000 -0.7572 -0.4692
END GEOMETRY

TASKS
  method = "UHF"
  propagatorTheoryCorrection=2
END TASKS

CONTROL
  readCoefficients=T      !read molecular orbitals generated with H2O.UHF.lowdin
  ionizeSpecies="E-ALPHA"
```

(continues on next page)

(continued from previous page)

```

ionizeM0=5
MOfractionOccupation=0.5
ptTransitionOperator=T
END CONTROL

```

Before running this calculation, rename the molecular orbitals file “H2O.UHF.vec” to “H2O.TOEP2.vec”

The output will include the summary of the P2 corrections to the partially ionized orbital

```

PROPAGATOR FORMALISM FOR SEVERAL FERMIONS SPECIES
ORDER OF CORRECTION = 2 + TRANSITION OPERATOR
The following articles must be cited:
-----
J. Chem. Phys. 127, 134106 (2007)
J. Chem. Phys. 137, 074105 (2012)
J. Chem. Phys. 138, 194108 (2013)
CPL coming soon
-----
SPECIES: E-ALPHA
-----

```

Orbital	KT (eV)	EP2 (eV)	P.S	SCS-EP2(eV)	P.S	SOS-
5	-10.364472	-10.768693	0.327967	-10.740363	0.357031	-10.
725240	0.369623					

```

-----

```

This summary includes the Koopmans’ (KT) and self-energy corrected (EP2) binding energy for the selected orbital. In addition, PT2 calculations with UHF reference include opposite-spin-scaled (SOS) and spin-component-scaled (SCS) results.

For the transition-operator results, the Pole Strength (P.S) value does not indicate the quality of the approximation.

PT3 FOR A PS-ATOM COMPLEX

A PT3 (third order *Propagator Theory*, *PT*) input on positronium chloride (PsCl), where the electrons and the positron are treated as quantum particles looks like this

```

GEOMETRY
e-(Cl) aug-cc-pvdz 0.0 0.0 0.0 addParticles=1 multiplicity=1
Cl      Dirac      0.0 0.0 0.0
E+      PsX-DZ     0.0 0.0 0.0
END GEOMETRY

TASKS
    method = "uhf"
    propagatorTheoryCorrection = 3
END TASKS

CONTROL
    ionizeSpecies = "e+"
    ionizeMO= 1
    ptJustOneOrbital=.T.
END CONTROL

```

APMO-PT3 calculations are performed when the option “propagatorTheoryCorrection=3” is present in the “TASKS” block. PT3 calculations only work with UHF as reference. Here, to save computational time, only the self-energy corrections to the positron occupied orbital will be performed. This is indicated by the “ionizeSpecies”, “ionizeMO” and “ptJustOneOrbital” entries in the CONTROL block.

The corresponding PT3 output will include the following summary

SUMMARY OF PROPAGATOR RESULTS FOR THE SPIN-ORBITAL: 1 OF SPECIES:E+		
Method	BE (eV)	Pole S.
KT	-3.918673	
EP2	-4.579523	0.975236
P3	-4.904336	0.954164
EP3	-4.830791	0.951763
OVGF-A	-4.915085	0.944111
OVGF-B	-4.945115	0.946181
OVGF-C	-4.900723	0.945407
REN-P3	-4.946967	0.952206

In this summary, KT is the eigenvalue of the orbital, which is the reference approximation to the particle binding energy according to Koopmans' theorem. The EP2 row corresponds to the second order propagator theory corrected binding energy.

Currently, openLowdin computes the third order corrections with six different formulas, denoted as P3, EP3, OVGf-A, OVGf-B, OVGf-C and REN-P3. Check J. Chem. Phys. 141, 114103 (2014) <https://doi.org/10.1063/1.4895043> for the full details.

P3 and EP3 refer to the partial and full third order propagator, respectively. The OVGf-x are the multicomponent extensions of the outer valence Green function methods, which are renormalized EP3 results. REN-P3 is the renormalized third order partial propagator (P3).

As in PT2 calculations, the Pole Strength serves as a quantity that validates the diagonal (pseudoparticle) approximation employed. A P.S value below 0.85 usually indicates that the diagonal approximation is not reliable.

If not all the corrections are desired, you can select in the "CONTROL" block the type of correction to be used by adding "ptP3Method=" with "P3","EP3","OVGF-A","OVGF-B","OVGF-C" or "REN-P3" as options.

As in MBPT calculations, the CONTROL option "mpFrozenCoreBoundary" allows the user to omit a number of occupied electronic molecular orbitals (core electrons). For other species, the number of core orbitals along with the number of active virtuals, can be controlled with the "INPUT_CI" block in the input.

NEGATIVE MUONS

QUANTUM DRUDE OSCILLATORS, QDOS

EXTERNAL POTENTIALS

MOLDEN EXAMPLES

Running the following input of positronic glycine

```
SYSTEM_DESCRIPTION='Gly.e+-molden'

GEOMETRY
e-(N) 6-311G      0.851078      0.379034      0.538699
e-(C) 6-311G      0.011668     -0.714576      1.101899
e-(C) 6-311G      0.016098     -0.668656      2.636109
e-(O) 6-311G      0.714088      0.201754      3.154209
e-(O) 6-311G     -0.685842     -1.520986      3.175689
e-(H) 6-311G      1.816418      0.263714      0.801099
e-(H) 6-311G      0.409998     -1.655016      0.751139
e-(H) 6-311G     -0.992842     -0.593136      0.726189
e-(H) 6-311G      0.527967      1.271782      0.883152
e-(H) 6-311G      0.787819      0.376813     -0.469282
e+ PSX-TZ         0.714088      0.201754      3.154209
e+ PSX-TZ     -0.685842     -1.520986      3.175689 addParticles=-1
N dirac          0.851078      0.379034      0.538699
C dirac          0.011668     -0.714576      1.101899
C dirac          0.016098     -0.668656      2.636109
O dirac          0.714088      0.201754      3.154209
O dirac     -0.685842     -1.520986      3.175689
H dirac          1.816418      0.263714      0.801099
H dirac          0.409998     -1.655016      0.751139
H dirac     -0.992842     -0.593136      0.726189
H dirac          0.527967      1.271782      0.883152
H dirac          0.787819      0.376813     -0.469282
END GEOMETRY

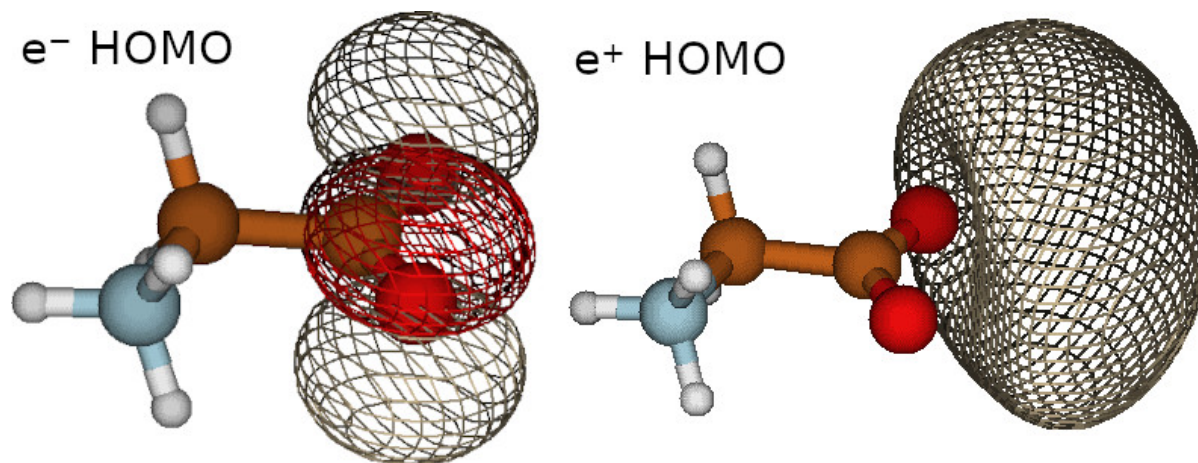
TASKS
      method = "RHF"
END TASKS

OUTPUTS
      moldenFile
END OUTPUTS
```

Produces two molden files, one for the electrons and one for the positron, their filenames are provided in the output.

```
-----
MOLDENFILE          1
                    for all species
FileNm: Gly.e+-molden.E-.molden
FileNm: Gly.e+-molden.E+.molden
-----
```

These files contain the the electronic and positronic orbitals. We can visualize these orbitals using the molden software (<https://www.theochem.ru.nl/molden/>), as observed in the following figure



32.1 Localized orbitals and fchk files

Localized orbitals are generated with the Ercale software (<https://github.com/susilehtola/erkale>), adding the following lines in the CONTROL block

```
SYSTEM_DESCRIPTION='Gly.e+-localize-molden'

GEOMETRY
e-(N) 6-311G      0.851078      0.379034      0.538699
e-(C) 6-311G      0.011668     -0.714576      1.101899
e-(C) 6-311G      0.016098     -0.668656      2.636109
e-(O) 6-311G      0.714088      0.201754      3.154209
e-(O) 6-311G     -0.685842     -1.520986      3.175689
e-(H) 6-311G      1.816418      0.263714      0.801099
e-(H) 6-311G      0.409998     -1.655016      0.751139
e-(H) 6-311G     -0.992842     -0.593136      0.726189
e-(H) 6-311G      0.527967      1.271782      0.883152
e-(H) 6-311G      0.787819      0.376813     -0.469282
e+ PSX-TZ         0.714088      0.201754      3.154209
e+ PSX-TZ        -0.685842     -1.520986      3.175689 addParticles=-1
N dirac           0.851078      0.379034      0.538699
C dirac           0.011668     -0.714576      1.101899
C dirac           0.016098     -0.668656      2.636109
O dirac           0.714088      0.201754      3.154209
O dirac          -0.685842     -1.520986      3.175689
```

(continues on next page)

(continued from previous page)

```

H dirac      1.816418    0.263714    0.801099
H dirac      0.409998   -1.655016    0.751139
H dirac     -0.992842   -0.593136    0.726189
H dirac      0.527967    1.271782    0.883152
H dirac      0.787819    0.376813   -0.469282
END GEOMETRY

TASKS
    method = "RHF"
END TASKS

CONTROL
    localizeOrbitals=.T.
    erkaleLocalizationMethod="MU"
END CONTROL

OUTPUTS
    moldenFile
END OUTPUTS

```

Here with “MU” we selected the Pipek-Mozay localization scheme using Mulliken charges. Check Erkale manual for a full list of the localization procedures available. To transfer the orbitals to Erkale, openLowdin generates fchk files. In the output, you will find the Erkale localization log.

```

-----
      FCHKFILE      1
  for all species
    FileName: Gly.e+-localize-molden.E-.fchk
    FileName: Gly.e+-localize-molden.E+.fchk
  -----

  ERKALE ORBITAL LOCALIZATION
  =====

  ERKALE - Localization from Hel, serial version.
  (c) Susi Lehtola, 2010-2016.

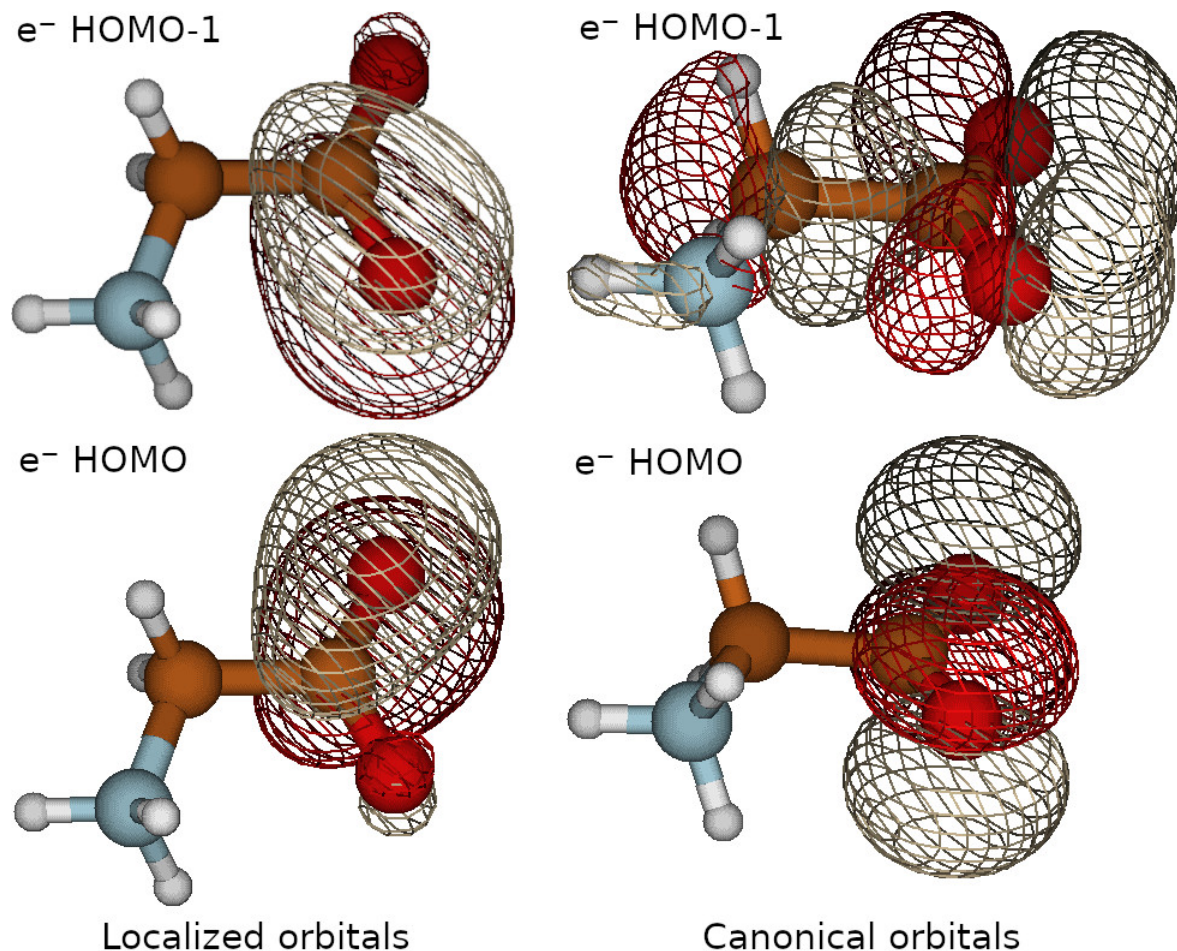
  [...]

  Localizing orbitals: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
  Initializing generalized Pipek-Mezey calculation with Mulliken charges... done.
  Initialization of Pipek-Mezey took 0.00 s
    iter      J      delta J      <G,G>
      1  1.154017e+01  1.517727e+00  1.174829e+00  0.00 s
      2  1.274872e+01  1.208552e+00  1.673368e+00  0.00 s
  [...]
      89  1.576852e+01  7.696281e-09  5.667527e-09  0.00 s
  Converged.
  Localization done in 0.06 s.

```

For this example, the localization procedure only affects the electronic orbitals, because there is only a single positronic

orbital. When localized orbitals are requested, openlowdin will generate the molden files with them. Check in the following figure a comparison between the non-localized (right) and localized (left) HOMO of positronic glycine



32.2 CI excited states

When we run a configuration interaction calculation we can generate molden files for the excited states. For example, in the *Positron covalent bond* (e+H-H-.CISDTQ-DZ.lowdin) example we added “state=2” to get the natural orbitals of the first excited state. In the output of that calculation we find

```
We are printing molden files for the CI states!
```

```
-----
MOLDENFILE      1
for all species
  FileName: e+H-H-.CISDTQ-DZ.E-ALPHA.molden
  FileName: e+H-H-.CISDTQ-DZ.E-BETA.molden
  FileName: e+H-H-.CISDTQ-DZ.E+.molden
-----
```

```
We are printing molden files for the CI states!
```

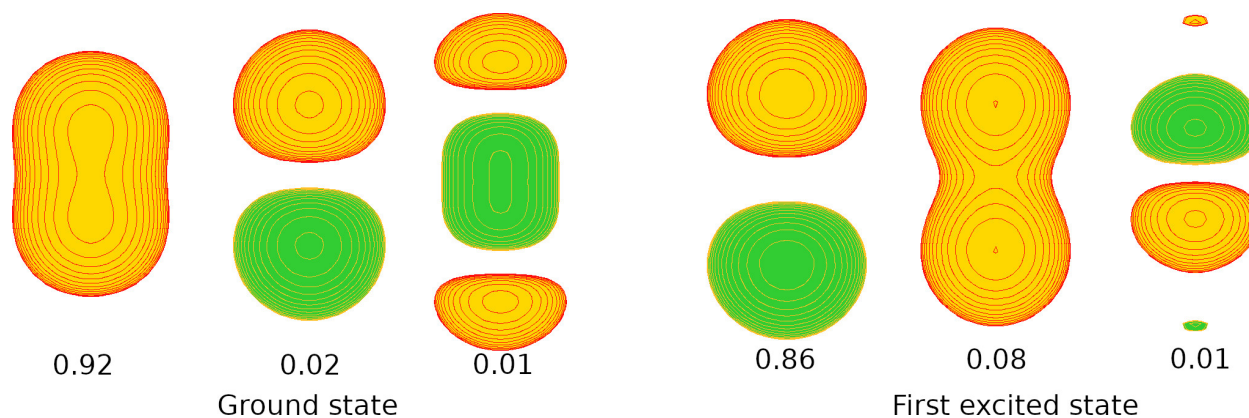
```
-----
MOLDENFILE      2
```

(continues on next page)

(continued from previous page)

```
for all species
for excited state:      2
  FileName: e+H-H-.CISDTQ-DZ.E-ALPHA-s2.molden
  FileName: e+H-H-.CISDTQ-DZ.E-BETA-s2.molden
  FileName: e+H-H-.CISDTQ-DZ.E+-s2.molden
-----
```

In the following figure we plot the positronic natural orbitals with the highest contributions to the ground (right) and first excited (left) states



CUBES EXAMPLES

With openLowdin, we can generate orbital and density Gaussian cube files (<https://h5cube-spec.readthedocs.io/en/latest/cubeformat.html>). Here we present an example for positronic formate

```
SYSTEM_DESCRIPTION='Positronic formate'

GEOMETRY
e-(C) cc-pVDZ          0.0000000 0.0000000 0.3157740
e-(H) cc-pVDZ          0.0000000 0.0000000 1.4510170
e-(O) aug-cc-pVDZ      0.0000000 1.1357680 -0.2091040
e-(O) aug-cc-pVDZ      0.0000000 -1.1357680 -0.2091040 addParticles=1
e+  PSX-DZ             0.0000000 1.1357680 -0.2091040
e+  PSX-DZ             0.0000000 -1.1357680 -0.2091040 addParticles=-1
C   dirac              0.0000000 0.0000000 0.3157740
H   dirac              0.0000000 0.0000000 1.4510170
O   dirac              0.0000000 1.1357680 -0.2091040
O   dirac              0.0000000 -1.1357680 -0.2091040
END GEOMETRY

TASKS
    method = "RHF"
END TASKS

OUTPUTS
    densityCube cubeSize=5 point1=0.0 0.0 0.0 species="E-" pointsPerDim=100
    densityCube cubeSize=20 point1=0.0 0.0 -2.5 species="E+" pointsPerDim=50
    orbitalCube cubeSize=15 center=0.0 0.0 -2.5 species="ALL" scanStep=0.5
    orbitalCube cubeSize=10 center=0.0 0.0 0.0 species="E-" orbital=11 scanStep=0.
→25
END OUTPUTS
```

With the first two lines in the OUTPUTS block, we request density cubes for electrons and positrons. Here, we use a different size for each cube, because the positronic density is more delocalized.

With the last two lines of the OUTPUTS block, we request orbitals cubes. When we select “species”=“ALL”, openLowdin will generate a cube with the HOMO of each species. We can also select a specific orbital, by selecting a species and an orbital number. In this example we requested the HOMO-1 of the formate anion.

We pass the option “center” or “point1” to define the origin of the cube, and we use the “pointsPerDim” or the “scanStep” options to adjust its resolution. “scanStep” sets the separation between grid points, whereas “pointsPerDim=N” sets the total number of points per side, such that the number of points in the cube is N^3 .

Therefore, running this calculation produces five cube files, with the filenames provided in the output.

```

-----
      DENSITYCUBE      1
    for species: E-
    cube size (a.u.):   9.44863
    cube center (a.u.): 0.000000  0.000000  0.000000
      No. steps:       100      100      100
      FileName: HCOOPs.HF.densOrbCube.E-.dens.cub
-----

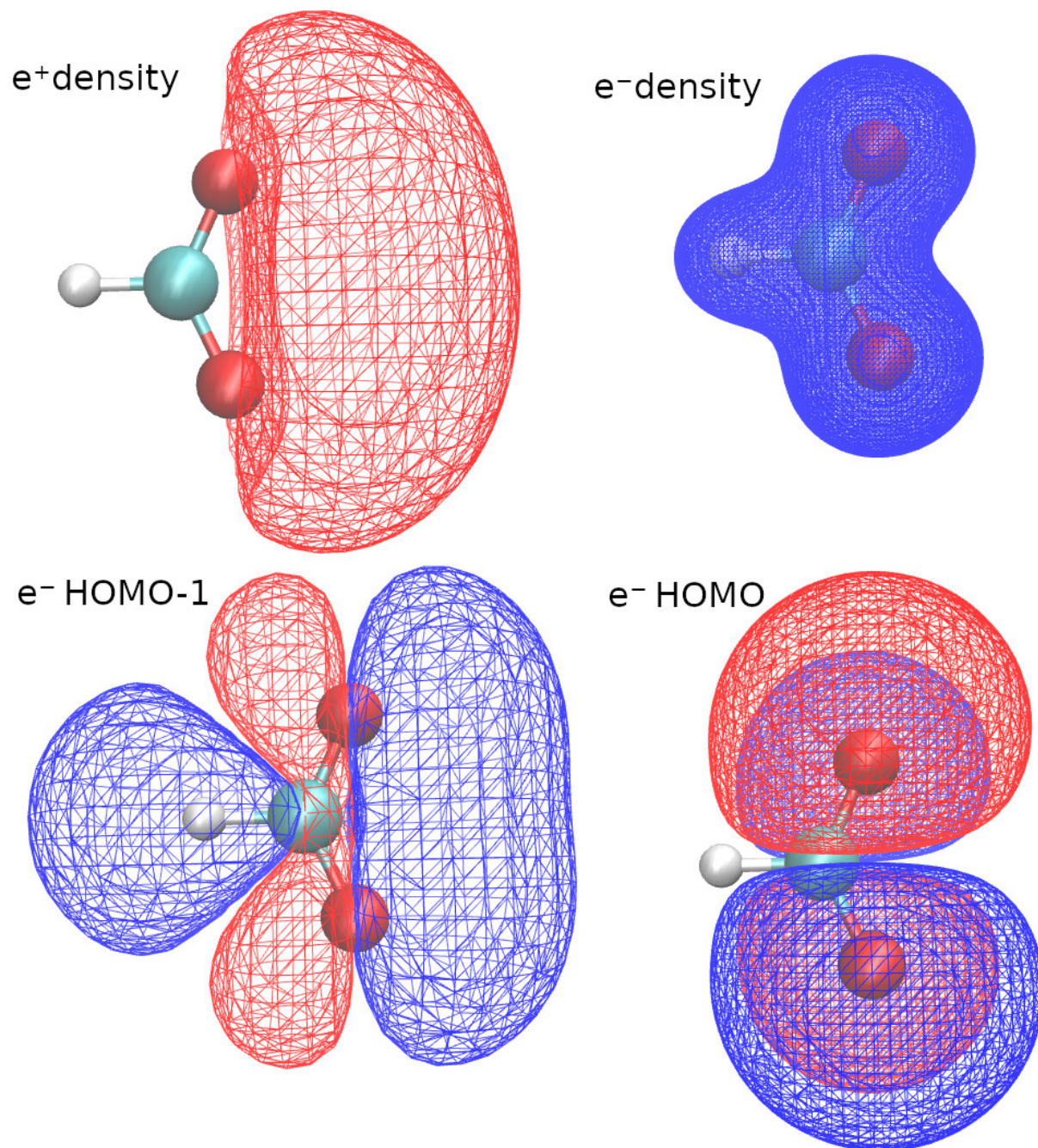
      DENSITYCUBE      2
    for species: E+
    cube size (a.u.):   37.79452
    cube center (a.u.): 0.000000  0.000000 -4.72432
      No. steps:        50       50       50
      FileName: HCOOPs.HF.densOrbCube.E+.dens.cub
-----

      ORBITALCUBE      3
    for all species
      for the highest occupied orbital
    cube size (a.u.):   28.34589
    cube center (a.u.): 0.000000  0.000000 -4.72432
      No. steps:        60       60       60
      FileName: HCOOPs.HF.densOrbCube.E-.orb12.cub
      FileName: HCOOPs.HF.densOrbCube.E+.orb1.cub
-----

      ORBITALCUBE      4
    for species: E-
      for orbital:      11
    cube size (a.u.):   18.89726
    cube center (a.u.): 0.000000  0.000000  0.000000
      No. steps:        80       80       80
      FileName: HCOOPs.HF.densOrbCube.E-.orb11.cub
-----

```

The VMD software (<https://www.ks.uiuc.edu/Research/vmd/>) is an excellent tool to visualize the cube files. Here are some VMD plots obtained from the positronic format cubes



We can also request cubes for localized orbitals. The keywords required are identical to those employed in the *Molden examples*.

33.1 Excited states

For configuration interaction calculations, we can generate cubes for excited state natural orbitals and excited state densities. For example, with the following CI singles calculation,

```
SYSTEM_DESCRIPTION='Positronic formate'

GEOMETRY
e-(C) cc-pVDZ      0.0000000  0.0000000  0.3157740
e-(H) cc-pVDZ      0.0000000  0.0000000  1.4510170
e-(O) aug-cc-pVDZ  0.0000000  1.1357680 -0.2091040
e-(O) aug-cc-pVDZ  0.0000000 -1.1357680 -0.2091040 addParticles=1
e+  PSX-DZ         0.0000000  1.1357680 -0.2091040
e+  PSX-DZ         0.0000000 -1.1357680 -0.2091040 addParticles=-1
C   dirac          0.0000000  0.0000000  0.3157740
H   dirac          0.0000000  0.0000000  1.4510170
O   dirac          0.0000000  1.1357680 -0.2091040
O   dirac          0.0000000 -1.1357680 -0.2091040
END GEOMETRY

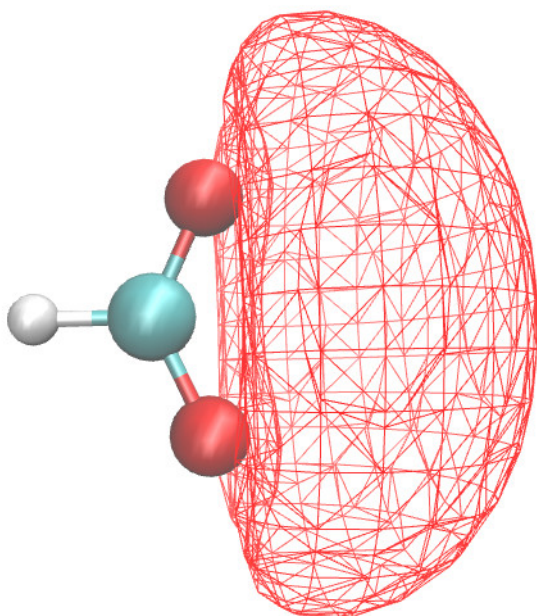
TASKS
      method = "UHF"
      configurationInteractionLevel="CIS"
END TASKS

CONTROL
      numberOfCIstates=2
      CIstatesToPrint=2
END CONTROL

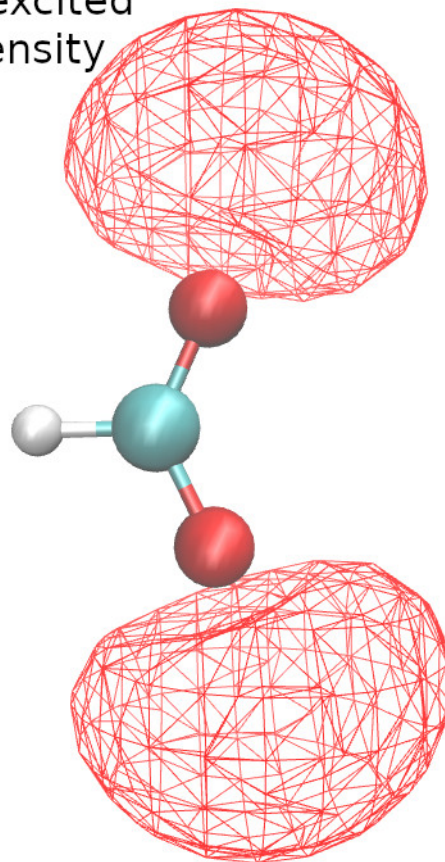
OUTPUTS
      densityCube  cubeSize=20  point1=0.0 0.0 -2.5 species="E+" pointsPerDim=50
↪state=1
      densityCube  cubeSize=20  point1=0.0 0.0 -2.5 species="E+" pointsPerDim=50
↪state=2
END OUTPUTS
```

we get the ground and first-excited state positronic densities.

ground state
e⁺density



1st excited
e⁺density



BIBLIOGRAPHY

- [1] Neil S. Ostlund Attila Szabo. *Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory*. Dover Publications, New York, United States, 1996. ISBN 9780486691862. doi:10.1119/1.1973756.
- [2] Roberto Flores-Moreno, Edwin Posada, Félix Moncada, Jonathan Romero, Jorge Charry, Manuel Díaz-Tinoco, Sergio A. González, Néstor F. Aguirre, and Andrés Reyes. LOWDIN: The any particle molecular orbital code. *Int. J. Quantum Chem*, 114(1):50–56, 2014. doi:10.1002/qua.24500.
- [3] Janus J. Eriksen, Tyler A. Anderson, J. Emiliano Deustua, Khaldoon Ghanem, Diptarka Hait, Mark R. Hoffmann, Seunghoon Lee, Daniel S. Levine, Ilias Magoulas, Jun Shen, Norm M. Tubman, K. Birgitta Whaley, Enhua Xu, Yuan Yao, Ning Zhang, Ali Alavi, Garnet Kin Lic Chan, Martin Head-Gordon, Wenjian Liu, Piotr Piecuch, Sandeep Sharma, Seiichiro L. Ten-No, C. J. Umrigar, and Jürgen Gauss. The Ground State Electronic Energy of Benzene. *J. Phys. Chem. Lett.*, 11(20):8922–8929, 2020. doi:10.1021/acs.jpclett.0c02621.
- [4] Norm M. Tubman, Joonho Lee, Tyler Y. Takeshita, Martin Head-Gordon, and K. Birgitta Whaley. A deterministic alternative to the full configuration interaction quantum Monte Carlo method. *J. Chem. Phys.*, 145(4):044112, 2016. doi:10.1063/1.4955109.